

循序渐进，学习开发 xv6-riscv

第 8 章 外部设备中断

汪辰



目录

01

RISC-V 中断的分类

02

中断编程中涉及的寄存器

03

RISC-V 中断处理流程

04

PLIC 介绍

05

基于中断方式实现串口输入



01

RISC-V 中断的分类

中断类型

- 本地 (Local) 中断
 - ✓ software interrupt
 - ✓ timer interrupt
- 全局 (Global) 中断
 - ✓ external interrupt

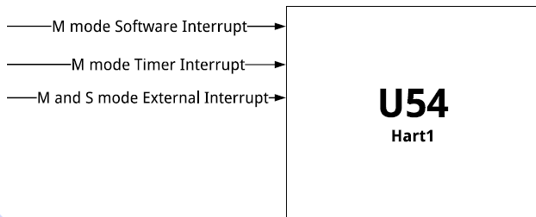


Table 32. Supervisor cause (scause) register values after trap.

Interrupt	Exception Code	Description
1	0	Reserved
1	1	Supervisor software interrupt
1	2-4	Reserved
1	5	Supervisor timer interrupt
1	6-8	Reserved
1	9	Supervisor external interrupt
1	10-12	Reserved
1	13	Counter-overflow interrupt
1	14-15	Reserved
1	≥16	Designated for platform use

中断类型

- 本地 (Local) 中断
 - ✓ software interrupt
 - ✓ timer interrupt
- 全局 (Global) 中断
 - ✓ external interrupt

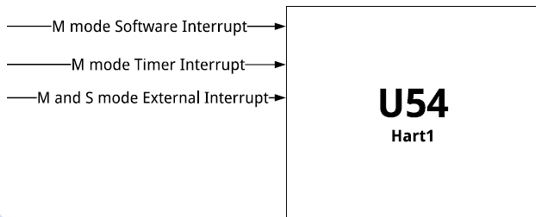


Table 32. Supervisor cause (scause) register values after trap.

Interrupt	Exception Code	Description
1	0	Reserved
1	1	Supervisor software interrupt
1	2-4	Reserved
1	5	Supervisor timer interrupt
1	6-8	Reserved
1	9	Supervisor external interrupt
1	10-12	Reserved
1	13	Counter-overflow interrupt
1	14-15	Reserved
1	≥16	Designated for platform use



02

中断编程中涉及的寄存器

Supervisor 模式下 Trap 相关 CSR 一览

寄存器	用途说明
stvec (Supervisor Trap Vector Base Address)	它保存异常发生时和处理器跳转地址相关的信息。
sepc (Supervisor Exception Program Counter)	当 trap 发生时, hart 会将发生 trap 所对应的指令的地址值 (pc) 保存在 sepc 中。
scause (Supervisor Cause)	当 trap 发生时, hart 会设置该寄存器通知我们 trap 发生的原因。
sip (Supervisor Interrupt Pending)	它列出目前已发生等待处理的中断类型。
sie (Supervisor Interrupt Enable)	用于进一步控制 (打开和关闭) software interrupt/timer interrupt/external interrupt
stval (Supervisor Trap Value)	它保存了 exception 发生时的附加信息: 譬如访问地址出错时的地址信息、或者执行非法指令时的指令本身, 对于其他异常, 它的值为 0。
sstatus (Supervisor Status)	用于跟踪和控制 hart 的当前操作状态 (特别地, 包括关闭和打开全局中断)。

Supervisor 模式下 Trap 相关 CSR 一览

寄存器	用途说明
stvec (Supervisor Trap Vector Base Address)	它保存异常发生时和处理器跳转地址相关的信息。
sepc (Supervisor Exception Program Counter)	当 trap 发生时, hart 会将发生 trap 所对应的指令的地址值 (pc) 保存在 sepc 中。
scause (Supervisor Cause)	当 trap 发生时, hart 会设置该寄存器通知我们 trap 发生的原因。
sip (Supervisor Interrupt Pending)	它列出目前已发生等待处理的中断。
sie (Supervisor Interrupt Enable)	用于进一步控制 (打开和关闭) software interrupt/timer interrupt/external interrupt
stval (Supervisor Trap Value)	它保存了 exception 发生时的附加信息: 譬如访问地址出错时的地址信息、或者执行非法指令时的指令本身, 对于其他异常, 它的值为 0。
sstatus (Supervisor Status)	用于跟踪和控制 hart 的当前操作状态 (特别地, 包括关闭和打开全局中断)。

Supervisor 模式下 Trap 相关 CSR 一览

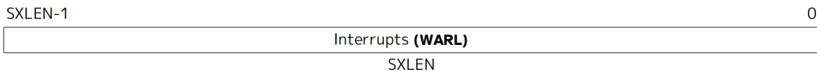


Figure 52. Supervisor interrupt-pending register (sip).

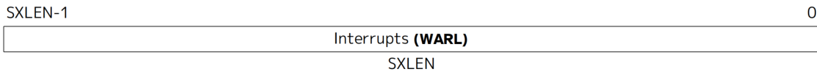


Figure 53. Supervisor interrupt-enable register (sie).

The RISC-V Instruction Set Manual Volume II , Chapter 12. Supervisor-Level ISA, Version 1.13

Supervisor 模式下 Trap 相关 CSR 一览

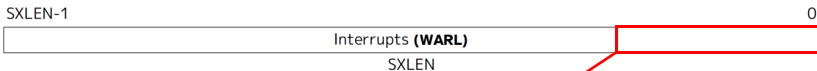


Figure 52. Supervisor interrupt-pending register (sip).

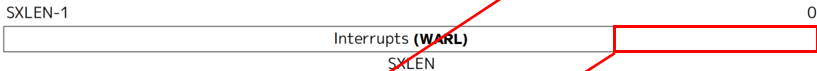


Figure 53. Supervisor interrupt-enable register (sie).

The RISC-V Instruction Set Manual Volume II , Chapter 12. Supervisor-Level ISA, Version 1.13

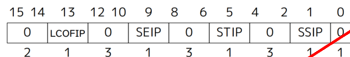


Figure 54. Standard portion (bits 15:0) of sip.

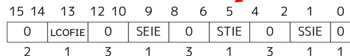
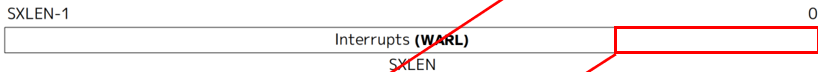
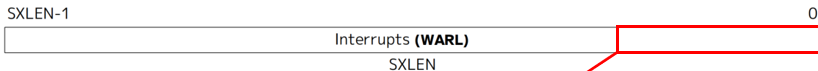


Figure 55. Standard portion (bits 15:0) of sie.

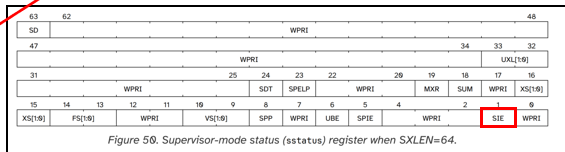
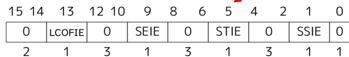
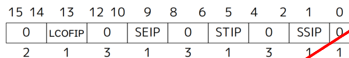
Table 32. Supervisor cause (scause) register values after trap.

Interrupt	Exception Code	Description
1	0	Reserved
1	1	Supervisor software interrupt
1	2-4	Reserved
1	5	Supervisor timer interrupt
1	6-8	Reserved
1	9	Supervisor external interrupt
1	10-12	Reserved
1	13	Counter-overflow interrupt
1	14-15	Reserved
1	≥16	Designated for platform use

Supervisor 模式下 Trap 相关 CSR 一览



The RISC-V Instruction Set Manual Volume II , Chapter 12. Supervisor-Level ISA, Version 1.13



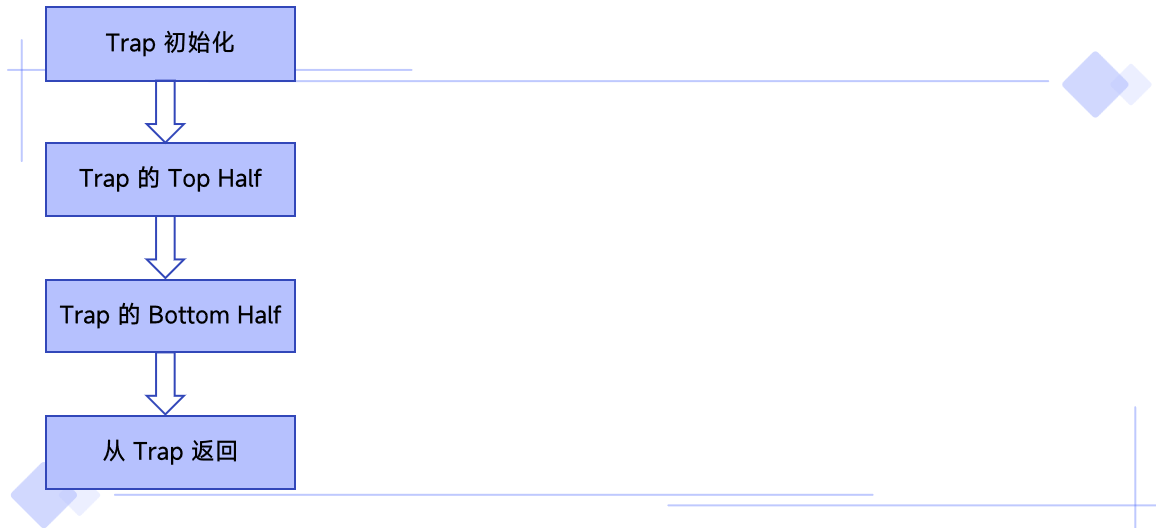
- 1 9 Supervisor external interrupt
- 1 10-12 Reserved
- 1 13 Counter-overflow interrupt
- 1 14-15 Reserved
- 1 ≥16 Designated for platform use

A decorative graphic on the left side of the slide. It features two concentric circles with a light blue glow. A thin grey arc with dots at its ends is positioned around the top and right of the circles. In the top right corner, there is a solid blue rectangle.

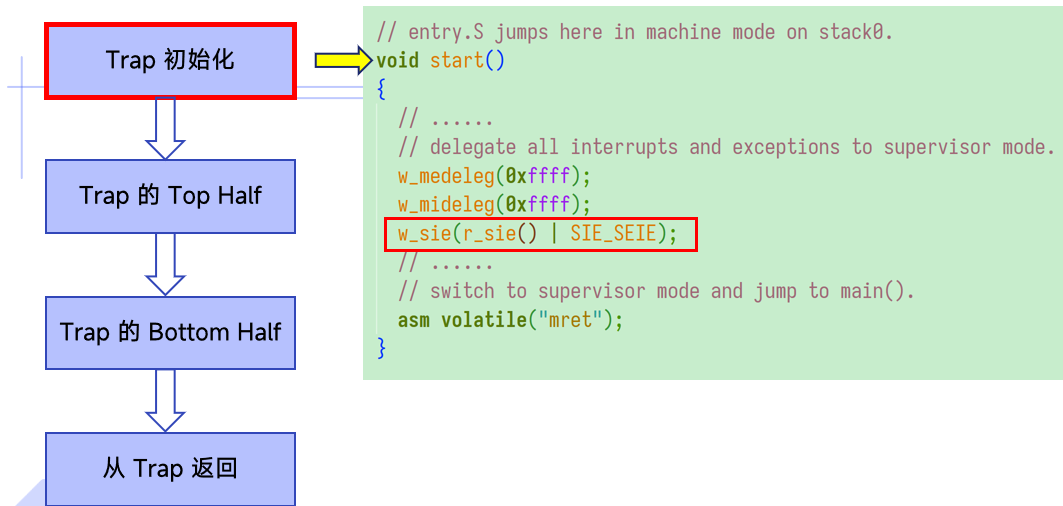
03

RISC-V 中断处理流程

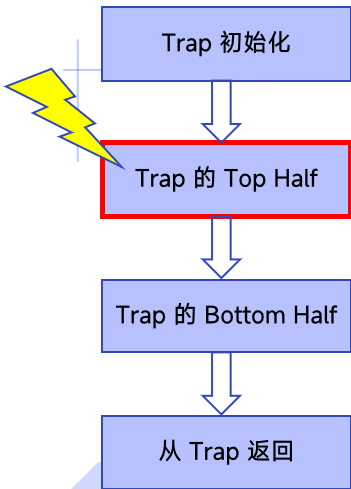
RISC-V 中断处理流程



RISC-V 中断处理流程

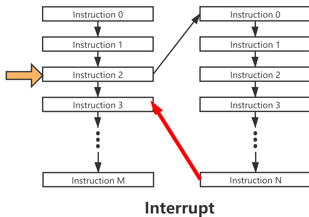


RISC-V 中断处理流程



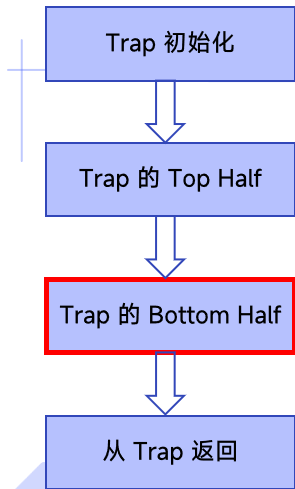
Trap 发生, Hart 自动执行如下状态转换

- 把 sstatus 的 SIE 值复制到 SPIE 中, 清除 sstatus 中的 SIE 标志位, 效果是 S 模式下的全局中断被禁止。
- 根据 trap 的种类设置 scause, 并根据需要为 stval 设置附加信息。
- 将 trap 发生之前的权限模式保存在 sstatus 的 SPP 域中, 再把 hart 权限模式更改为 Supervisor 模式 (前提是我们已经通过 medeleg 和 mideleg 实现了委托, 否则默认切换到 Machine 模式)。
- 设置 sepc。对于 interrupt, 它指向被中断的指令的下一条指令的位置。



- PC 被设置为 stvec。

RISC-V 中断处理流程



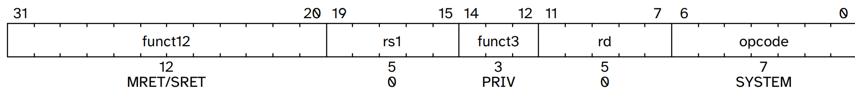
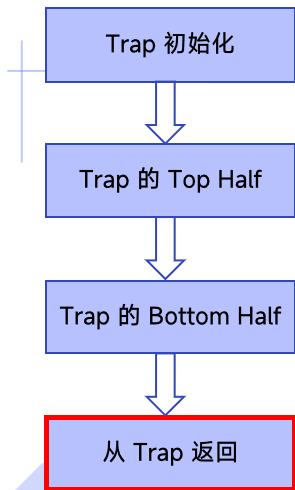
```
.globl kerneltrap
.globl kernelvec
.align 4
kernelvec:
    # make room to save registers.
    addi sp, sp, -256
    # save caller-saved registers.
    sd ra, 0(sp)
    # .....
    sd t6, 240(sp)

    # call the C trap handler in trap.c
    call kerneltrap

    # restore registers.
    ld ra, 0(sp)
    # .....
    ld t6, 240(sp)
    addi sp, sp, 256

    # return to whatever we were doing in the kernel.
    sret
```

RISC-V 中断处理流程



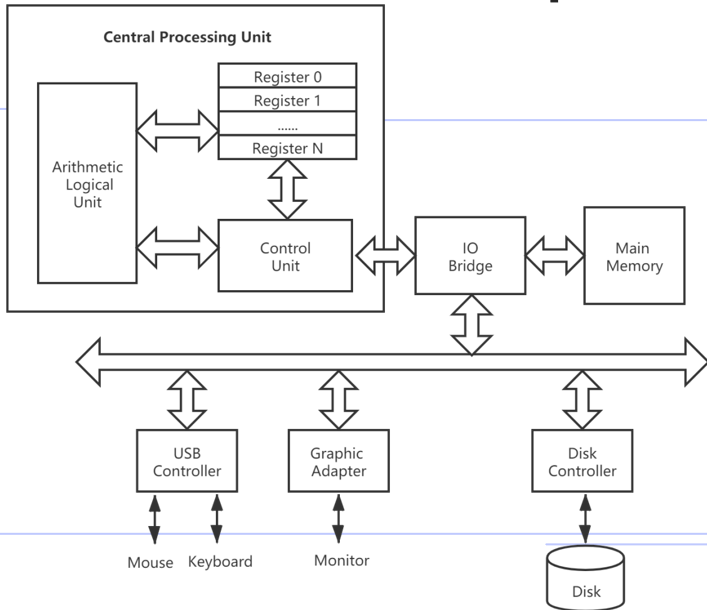
- 针对不同权限级别下如何退出 trap 有各自的返回指令 xRET ($x = M/S$)
- 以在 S 模式下执行 sret 指令为例，会执行如下操作：
 - ✓ 返回后 Hart 的权限级别 = sstatus.SPP;
 - ✓ sstatus.SPP = 0;
 - ✓ sstatus.SIE = sstatus.SPIE;
 - ✓ sstatus.SPIE = 1
 - ✓ pc = sepc



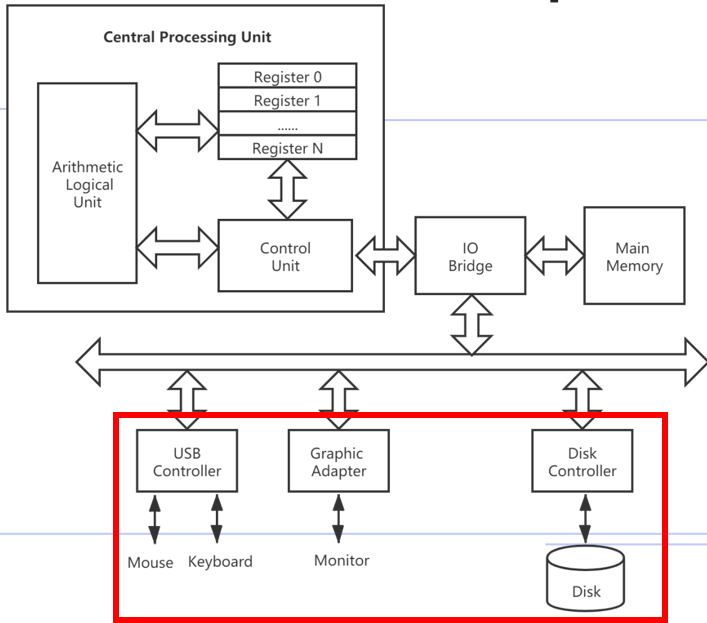
04

PLIC 介绍

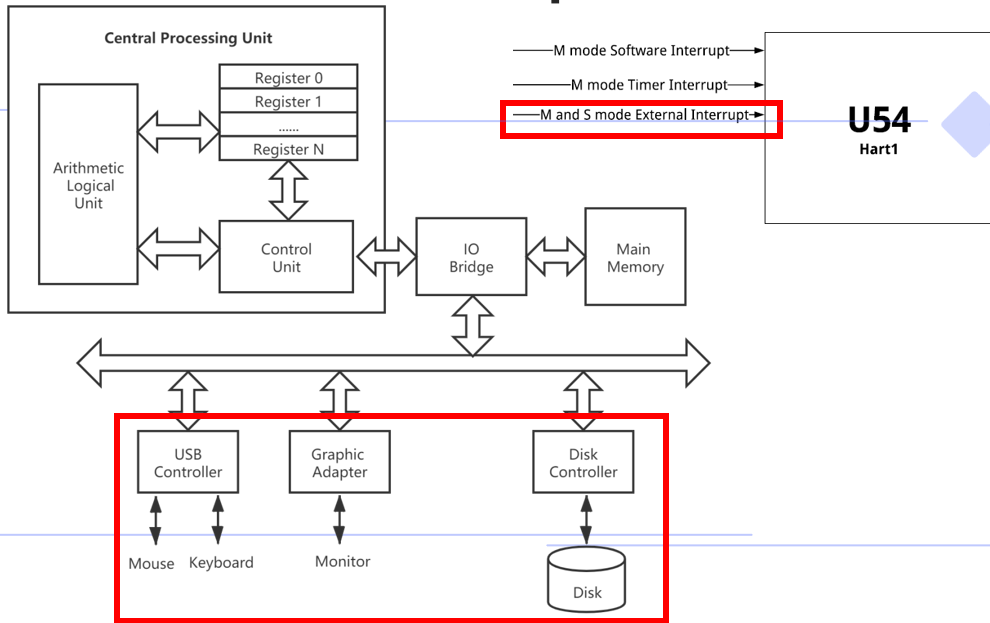
外部中断 (external interrupt)



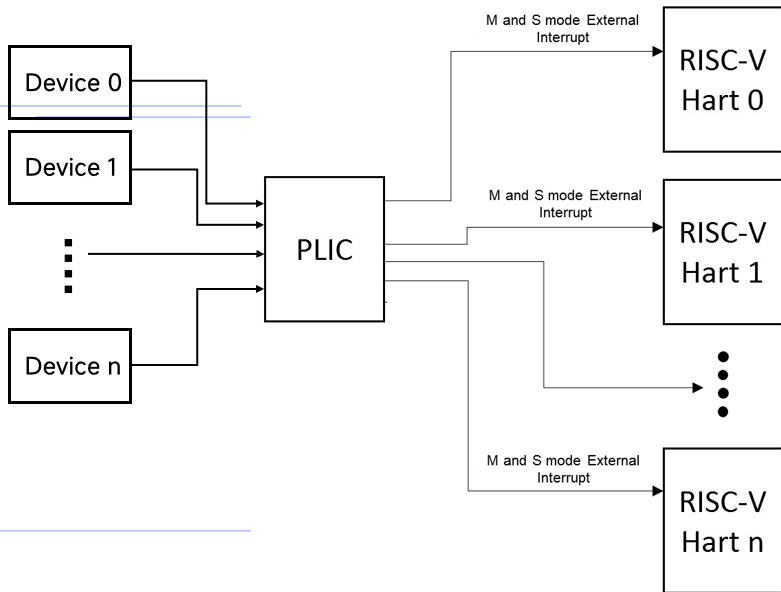
外部中断 (external interrupt)



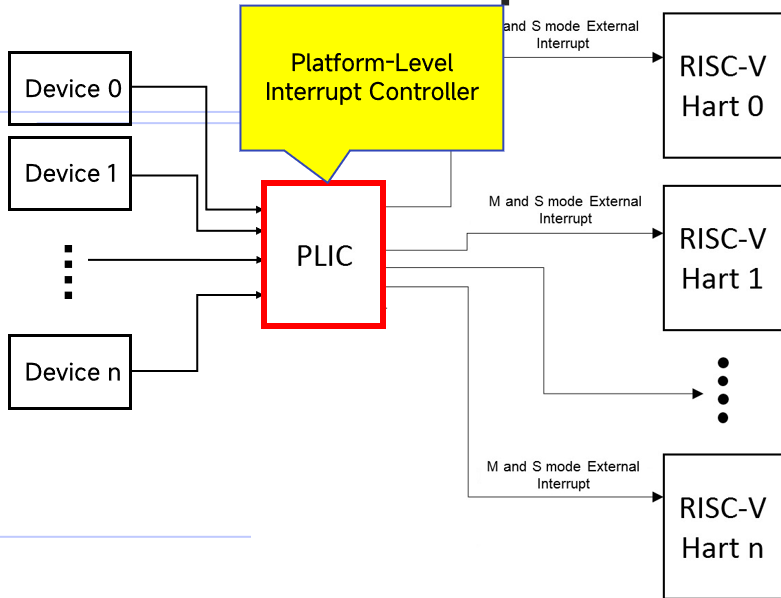
外部中断 (external interrupt)



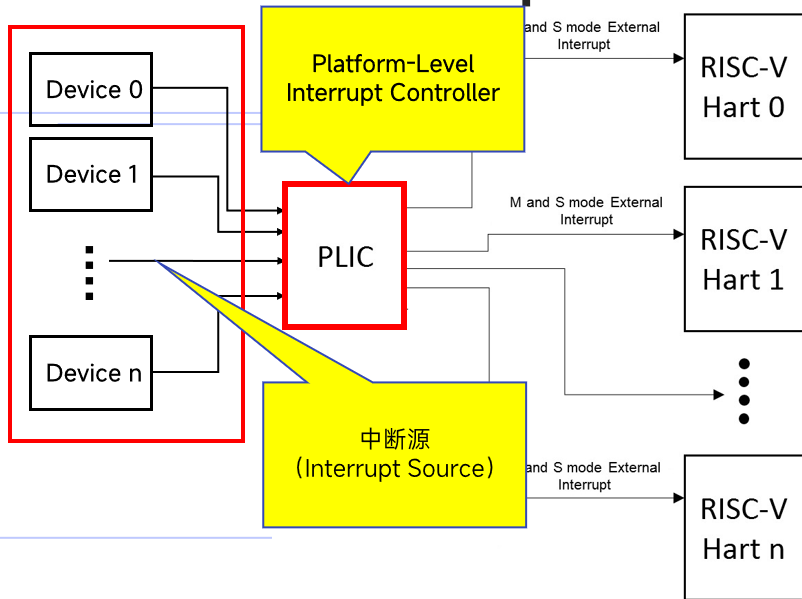
外部中断 (external interrupt)



外部中断 (external interrupt)



外部中断 (external interrupt)



PLIC Interrupt Source

<https://github.com/qemu/qemu/blob/master/include/hw/riscv/virt.h>

```
enum {  
    UART0_IRQ = 10,  
    RTC_IRQ = 11,  
    VIRTIO_IRQ = 1, /* 1 to 8 */  
    VIRTIO_COUNT = 8,  
    PCIE_IRQ = 0x20, /* 32 to 35 */  
    IOMMU_SYS_IRQ = 0x24, /* 36-39 */  
    VIRT_PLATFORM_BUS_IRQ = 64, /* 64 to 95 */  
};
```

PLIC Interrupt Source

<https://github.com/qemu/qemu/blob/master/include/hw/riscv/virt.h>

```
enum {  
    UART0_IRQ = 10,  
    RTC_IRQ = 11,  
    VIRTIO_IRQ = 1, /* 1 to 8 */  
    VIRTIO_COUNT = 8,  
    PCIE_IRQ = 0x20, /* 32 to 35 */  
    IOMMU_SYS_IRQ = 0x24, /* 36-39 */  
    VIRT_PLATFORM_BUS_IRQ = 64, /* 64 to 95 */  
};
```

```
#define UART0_IRQ 10
```

PLIC 编程接口 - CSR (1)

PLIC Memory Map

```
base + 0x000000: Reserved (interrupt source 0 does not exist)
base + 0x000004: Interrupt source 1 priority
base + 0x000008: Interrupt source 2 priority
...
base + 0x000FFC: Interrupt source 1023 priority
base + 0x001000: Interrupt Pending bit 0-31
base + 0x00107C: Interrupt Pending bit 992-1023
...
base + 0x002000: Enable bits for sources 0-31 on context 0
base + 0x002004: Enable bits for sources 32-63 on context 0
...
base + 0x00207C: Enable bits for sources 992-1023 on context 0
base + 0x002080: Enable bits for sources 0-31 on context 1
base + 0x002084: Enable bits for sources 32-63 on context 1
...
base + 0x0020FC: Enable bits for sources 992-1023 on context 1
base + 0x002100: Enable bits for sources 0-31 on context 2
base + 0x002104: Enable bits for sources 32-63 on context 2
...
base + 0x00217C: Enable bits for sources 992-1023 on context 2
...
base + 0x1F1F80: Enable bits for sources 0-31 on context 15871
base + 0x1F1F84: Enable bits for sources 32-63 on context 15871
base + 0x1F1FFC: Enable bits for sources 992-1023 on context 15871
...
base + 0x1FFFFC: Reserved
base + 0x200000: Priority threshold for context 0
base + 0x200004: Claim/complete for context 0
base + 0x200008: Reserved
...
base + 0x200FFC: Reserved
base + 0x201000: Priority threshold for context 1
base + 0x201004: Claim/complete for context 1
...
base + 0x3FFF000: Priority threshold for context 15871
base + 0x3FFF004: Claim/complete for context 15871
base + 0x3FFF008: Reserved
...
base + 0x3FFFFFFC: Reserved
```

- **RISC-V 规范规定，PLIC 的寄存器编址采用内存映射 (memory map) 方式。每个寄存器的宽度为 32-bit。**
- Memory Map 大小: 取决于配置的 CPU 数量和中断源数量，最大为 0x40000000 (64 MB)
- 基地址 (base): 由硬件平台自己定义，QEMU 的 virt 设备设定的 PLIC 基地址参考了 FU540-C000，为: 0x0c000000

PLIC 编程接口 - CSR (1)

PLIC Memory Map

```
base + 0x000000: Reserved (interrupt source 0 does not exist)
base + 0x000004: Interrupt source 1 priority
base + 0x000008: Interrupt source 2 priority
...
base + 0x000FFC: Interrupt source 1023 priority
base + 0x001000: Interrupt Pending bit 0-31
base + 0x00107C: Interrupt Pending bit 992-1023
...
base + 0x002000: Enable bits for sources 0-31 on context 0
base + 0x002004: Enable bits for sources 32-63 on context 0
...
base + 0x00207C: Enable bits for sources 992-1023 on context 0
base + 0x002080: Enable bits for sources 0-31 on context 1
base + 0x002084: Enable bits for sources 32-63 on context 1
...
base + 0x0020FC: Enable bits for sources 992-1023 on context 1
base + 0x002100: Enable bits for sources 0-31 on context 2
base + 0x002104: Enable bits for sources 32-63 on context 2
...
base + 0x00217C: Enable bits for sources 992-1023 on context 2
...
base + 0x1F1F80: Enable bits for sources 0-31 on context 15871
base + 0x1F1F84: Enable bits for sources 32-63 on context 15871
base + 0x1F1FFC: Enable bits for sources 992-1023 on context 15871
...
base + 0x1FFFFC: Reserved
base + 0x200000: Priority threshold for context 0
base + 0x200004: Claim/complete for context 0
base + 0x200008: Reserved
...
base + 0x200FFC: Reserved
base + 0x201000: Priority threshold for context 1
base + 0x201004: Claim/complete for context 1
...
base + 0x3FFF00: Priority threshold for context 15871
base + 0x3FFF04: Claim/complete for context 15871
base + 0x3FFF08: Reserved
...
base + 0x3FFFFFFC: Reserved
```

- RISC-V 规范规定，PLIC 的寄存器编址采用内存映射（memory map）方式。每个寄存器的宽度为 32-bit。
- **Memory Map 大小: 取决于配置的 CPU 数量和中断源数量，最大为 0x4000000 (64 MB)**
- 基地址（base）：由硬件平台自己定义，QEMU 的 virt 设备设置的 PLIC 基地址参考了 FU540-C000，为：0x0c000000

PLIC 编程接口 - CSR (2)

PLIC Memory Map

```
base + 0x000000: Reserved (interrupt source 0 does not exist)
base + 0x000004: Interrupt source 1 priority
base + 0x000008: Interrupt source 2 priority
...
base + 0x000FFC: Interrupt source 1023 priority
base + 0x001000: Interrupt Pending bit 0-31
base + 0x00107C: Interrupt Pending bit 992-1023
...
base + 0x002000: Enable bits for sources 0-31 on context 0
base + 0x002004: Enable bits for sources 32-63 on context 0
...
base + 0x00207C: Enable bits for sources 992-1023 on context 0
base + 0x002080: Enable bits for sources 0-31 on context 1
base + 0x002084: Enable bits for sources 32-63 on context 1
...
base + 0x0020FC: Enable bits for sources 992-1023 on context 1
base + 0x002100: Enable bits for sources 0-31 on context 2
base + 0x002104: Enable bits for sources 32-63 on context 2
...
base + 0x00217C: Enable bits for sources 992-1023 on context 2
...
base + 0x1F1F80: Enable bits for sources 0-31 on context 15871
base + 0x1F1F84: Enable bits for sources 32-63 on context 15871
base + 0x1F1FFC: Enable bits for sources 992-1023 on context 15871
...
base + 0x1FFFC: Reserved
base + 0x200000: Priority threshold for context 0
base + 0x200004: Claim/complete for context 0
base + 0x200008: Reserved
...
base + 0x200FFC: Reserved
base + 0x201000: Priority threshold for context 1
base + 0x201004: Claim/complete for context 1
...
base + 0x3FFF00: Priority threshold for context 15871
base + 0x3FFF04: Claim/complete for context 15871
base + 0x3FFF08: Reserved
...
base + 0x3FFFFFFC: Reserved
```

- RISC-V 规范规定, PLIC 的寄存器编址采用内存映射 (memory map) 方式。每个寄存器的宽度为 32-bit。
- Memory Map 大小: 取决于配置的 CPU 数量和中断源数量, 最大为 0x4000000 (64 MB)
- 基地址 (base): 由硬件平台自己定义, QEMU 的 virt 设备设定的 PLIC 基地址参考了 FU540-C000, 为: **0x0c000000**

<https://github.com/qemu/qemu/blob/master/hw/riscv/virt.c>

```
static const MemMapEntry virt_memmap[] = {
    [VIRT_DEBUG] = { 0x0, 0x100 },
    [VIRT_MROM] = { 0x1000, 0xf000 },
    [VIRT_TEST] = { 0x100000, 0x1000 },
    [VIRT_RTC] = {
    [VIRT_CLINT] = {
    [VIRT_ACLINT_SSWI] = {
    [VIRT_PCIE_PIO] = {
    [VIRT_IOMMU_SYS] = { 0x3010000, 0x1000 },
    [VIRT_PLATFORM_BUS] = { 0x4000000, 0x2000000 },
    [VIRT_PLIC] = { 0xc000000, VIRT_PLIC_SIZE(VIRT_CPUS_MAX * 2) },
    [VIRT_APLIC_M] = { 0xc000000, APLIC_SIZE(VIRT_CPUS_MAX) },
    [VIRT_APLIC_S] = { 0xd000000, APLIC_SIZE(VIRT_CPUS_MAX) },
    [VIRT_UART0] = { 0x10000000, 0x100 },
    [VIRT_VIRTIO] = { 0x10001000, 0x1000 },
    [VIRT_FW_CFG] = { 0x10100000, 0x18 },
    [VIRT_FLASH] = { 0x20000000, 0x4000000 },
    [VIRT_IMSIC_M] = { 0x24000000, VIRT_IMSIC_MAX_SIZE },
    [VIRT_IMSIC_S] = { 0x28000000, VIRT_IMSIC_MAX_SIZE },
    [VIRT_PCIE_ECAM] = { 0x30000000, 0x10000000 },
    [VIRT_PCIE_MMIO] = { 0x40000000, 0x40000000 },
    [VIRT_DRAM] = { 0x80000000, 0x0 },
};
```

#define PLIC 0x0c000000L

PLIC 编程接口 - 中断源 Priority 寄存器 (1)

PLIC Memory Map

```
base + 0x000000: Reserved (interrupt source 0 does not exist)
base + 0x000004: Interrupt source 1 priority
base + 0x000008: Interrupt source 2 priority
...
base + 0x000FFC: Interrupt source 1023 priority
base + 0x001000: Interrupt Pending bit 0-31
base + 0x00107C: Interrupt Pending bit 992-1023
...
base + 0x002000: Enable bits for sources 0-31 on context 0
base + 0x002004: Enable bits for sources 32-63 on context 0
...
base + 0x00207C: Enable bits for sources 992-1023 on context 0
base + 0x002080: Enable bits for sources 0-31 on context 1
base + 0x002084: Enable bits for sources 32-63 on context 1
...
base + 0x0020FC: Enable bits for sources 992-1023 on context 1
base + 0x002100: Enable bits for sources 0-31 on context 2
base + 0x002104: Enable bits for sources 32-63 on context 2
...
base + 0x00217C: Enable bits for sources 992-1023 on context 2
...
base + 0x1F1F80: Enable bits for sources 0-31 on context 15871
base + 0x1F1F84: Enable bits for sources 32-63 on context 15871
base + 0x1F1FFC: Enable bits for sources 992-1023 on context 15871
...
base + 0x1FFFC: Reserved
base + 0x200000: Priority threshold for context 0
base + 0x200004: Claim/complete for context 0
base + 0x200008: Reserved
...
base + 0x200FFC: Reserved
base + 0x201000: Priority threshold for context 1
base + 0x201004: Claim/complete for context 1
...
base + 0x3FFF00: Priority threshold for context 15871
base + 0x3FFF04: Claim/complete for context 15871
base + 0x3FFF08: Reserved
...
base + 0x3FFFFFFC: Reserved
```

- 每个 PLIC 中断源对应一个 Priority 寄存器，用于配置该中断源的优先级。每个寄存器的宽度是 4 字节 (32 位)
- 最多 1024 个 中断源，0 号中断源不可用。
- 寄存器映射地址 = $\text{BASE} + (\text{中断源 ID}) * 4$
- QEMU-virt 支持 7 个优先级。0 表示对该中断源禁用中断。其余优先级，1 最低，7 最高。
- 如果两个中断源优先级相同，则根据中断源的 ID 值进一步区分优先级，ID 值越小的优先级越高。

PLIC 编程接口 - 中断源 Priority 寄存器 (2)

PLIC Memory Map

```
base + 0x000000: Reserved (interrupt source 0 does not exist)
base + 0x000004: Interrupt source 1 priority
base + 0x000008: Interrupt source 2 priority
...
base + 0x000FFC: Interrupt source 1023 priority
base + 0x001000: Interrupt Pending bit 0-31
base + 0x00107C: Interrupt Pending bit 992-1023
...
base + 0x002000: Enable bits for sources 0-31 on context 0
base + 0x002004: Enable bits for sources 32-63 on context 0
...
base + 0x00207C: Enable bits for sources 992-1023 on context 0
base + 0x002080: Enable bits for sources 0-31 on context 1
base + 0x002084: Enable bits for sources 32-63 on context 1
...
base + 0x0020FC: Enable bits for sources 992-1023 on context 1
base + 0x002100: Enable bits for sources 0-31 on context 2
base + 0x002104: Enable bits for sources 32-63 on context 2
...
base + 0x00217C: Enable bits for sources 992-1023 on context 2
...
base + 0x1F1F80: Enable bits for sources 0-31 on context 15871
base + 0x1F1F84: Enable bits for sources 32-63 on context 15871
base + 0x1F1FFC: Enable bits for sources 992-1023 on context 15871
...
base + 0x1FFFC: Reserved
base + 0x200000: Priority threshold for context 0
base + 0x200004: Claim/complete for context 0
base + 0x200008: Reserved
...
base + 0x200FFC: Reserved
base + 0x201000: Priority threshold for context 1
base + 0x201004: Claim/complete for context 1
...
base + 0x3FFF00: Priority threshold for context 15871
base + 0x3FFF04: Claim/complete for context 15871
base + 0x3FFF08: Reserved
...
base + 0x3FFFFFC: Reserved
```

- 每个 PLIC 中断源对应一个 Priority 寄存器，用于配置该中断源的优先级。每个寄存器的宽度是 4 字节（32 位）
- **最多 1024 个中断源，0 号中断源不可用。**
- 寄存器映射地址 = $\text{BASE} + (\text{中断源 ID}) * 4$
- QEMU-virt 支持 7 个优先级。0 表示对该中断源禁用中断。其余优先级，1 最低，7 最高。
- 如果两个中断源优先级相同，则根据中断源的 ID 值进一步区分优先级，ID 值越小的优先级越高。

PLIC 编程接口 - 中断源 Priority 寄存器 (3)

PLIC Memory Map

```
base + 0x000000: Reserved (interrupt source 0 does not exist)
base + 0x000004: Interrupt source 1 priority
base + 0x000008: Interrupt source 2 priority
...
base + 0x000FFC: Interrupt source 1023 priority
base + 0x001000: Interrupt Pending bit 0-31
base + 0x00107C: Interrupt Pending bit 992-1023
...
base + 0x002000: Enable bits for sources 0-31 on context 0
base + 0x002004: Enable bits for sources 32-63 on context 0
...
base + 0x00207C: Enable bits for sources 992-1023 on context 0
base + 0x002080: Enable bits for sources 0-31 on context 1
base + 0x002084: Enable bits for sources 32-63 on context 1
...
base + 0x0020FC: Enable bits for sources 992-1023 on context 1
base + 0x002100: Enable bits for sources 0-31 on context 2
base + 0x002104: Enable bits for sources 32-63 on context 2
...
base + 0x00217C: Enable bits for sources 992-1023 on context 2
...
base + 0x1F1F80: Enable bits for sources 0-31 on context 15871
base + 0x1F1F84: Enable bits for sources 32-63 on context 15871
base + 0x1F1FFC: Enable bits for sources 992-1023 on context 15871
...
base + 0x1FFFC: Reserved
base + 0x200000: Priority threshold for context 0
base + 0x200004: Claim/complete for context 0
base + 0x200008: Reserved
...
base + 0x200FFC: Reserved
base + 0x201000: Priority threshold for context 1
base + 0x201004: Claim/complete for context 1
...
base + 0x3FFF00: Priority threshold for context 15871
base + 0x3FFF04: Claim/complete for context 15871
base + 0x3FFF08: Reserved
...
base + 0x3FFFFFFC: Reserved
```

- 每个 PLIC 中断源对应一个 Priority 寄存器，用于配置该中断源的优先级。每个寄存器的宽度是 4 字节（32 位）
- 最多 1024 个 中断源，0 号中断源不可用。
- **寄存器映射地址 = BASE + (中断源 ID) * 4**
- QEMU-virt 支持 7 个优先级。0 表示对该中断源禁用中断。其余优先级，1 最低，7 最高。
- 如果两个中断源优先级相同，则根据中断源的 ID 值进一步区分优先级，ID 值越小的优先级越高。

PLIC 编程接口 - 中断源 Priority 寄存器 (4)

PLIC Memory Map

```
base + 0x000000: Reserved (interrupt source 0 does not exist)
base + 0x000004: Interrupt source 1 priority
base + 0x000008: Interrupt source 2 priority
...
base + 0x000FFC: Interrupt source 1023 priority
base + 0x001000: Interrupt Pending bit 0-31
base + 0x00107C: Interrupt Pending bit 992-1023
...
base + 0x002000: Enable bits for sources 0-31 on context 0
base + 0x002004: Enable bits for sources 32-63 on context 0
...
base + 0x00207C: Enable bits for sources 992-1023 on context 0
base + 0x002080: Enable bits for sources 0-31 on context 1
base + 0x002084: Enable bits for sources 32-63 on context 1
...
base + 0x0020FC: Enable bits for sources 992-1023 on context 1
base + 0x002100: Enable bits for sources 0-31 on context 2
base + 0x002104: Enable bits for sources 32-63 on context 2
...
base + 0x00217C: Enable bits for sources 992-1023 on context 2
...
base + 0x1F1F80: Enable bits for sources 0-31 on context 15871
base + 0x1F1F84: Enable bits for sources 32-63 on context 15871
base + 0x1F1FFC: Enable bits for sources 992-1023 on context 15871
...
base + 0x1FFFC: Reserved
base + 0x200000: Priority threshold for context 0
base + 0x200004: Claim/complete for context 0
base + 0x200008: Reserved
...
base + 0x200FFC: Reserved
base + 0x201000: Priority threshold for context 1
base + 0x201004: Claim/complete for context 1
...
base + 0x3FFF00: Priority threshold for context 15871
base + 0x3FFF04: Claim/complete for context 15871
base + 0x3FFF08: Reserved
...
base + 0x3FFFFFC: Reserved
```

- 每个 PLIC 中断源对应一个 Priority 寄存器，用于配置该中断源的优先级。每个寄存器的宽度是 4 字节 (32 位)
- 最多 1024 个 中断源，0 号中断源不可用。
- 寄存器映射地址 = $\text{BASE} + (\text{中断源 ID}) * 4$
- **QEMU-virt 支持 7 个优先级。0 表示对该中断源禁用中断。其余优先级，1 最低，7 最高。**
- 如果两个中断源在同一个 PLIC 中，需要进一步区分

```
void plicinit(void)
{
    *(uint32*)(PLIC + UART0_IRQ*4) = 1;
}
```

PLIC 编程接口 - 中断源 Priority 寄存器 (5)

PLIC Memory Map

```
base + 0x000000: Reserved (interrupt source 0 does not exist)
base + 0x000004: Interrupt source 1 priority
base + 0x000008: Interrupt source 2 priority
...
base + 0x000FFC: Interrupt source 1023 priority
base + 0x001000: Interrupt Pending bit 0-31
base + 0x00107C: Interrupt Pending bit 992-1023
...
base + 0x002000: Enable bits for sources 0-31 on context 0
base + 0x002004: Enable bits for sources 32-63 on context 0
...
base + 0x00207C: Enable bits for sources 992-1023 on context 0
base + 0x002080: Enable bits for sources 0-31 on context 1
base + 0x002084: Enable bits for sources 32-63 on context 1
...
base + 0x0020FC: Enable bits for sources 992-1023 on context 1
base + 0x002100: Enable bits for sources 0-31 on context 2
base + 0x002104: Enable bits for sources 32-63 on context 2
...
base + 0x00217C: Enable bits for sources 992-1023 on context 2
...
base + 0x1F1F80: Enable bits for sources 0-31 on context 15871
base + 0x1F1F84: Enable bits for sources 32-63 on context 15871
base + 0x1F1FFC: Enable bits for sources 992-1023 on context 15871
...
base + 0x1FFFC: Reserved
base + 0x200000: Priority threshold for context 0
base + 0x200004: Claim/complete for context 0
base + 0x200008: Reserved
...
base + 0x200FFC: Reserved
base + 0x201000: Priority threshold for context 1
base + 0x201004: Claim/complete for context 1
...
base + 0x3FFF00: Priority threshold for context 15871
base + 0x3FFF04: Claim/complete for context 15871
base + 0x3FFF08: Reserved
...
base + 0x3FFFFFFC: Reserved
```

- 每个 PLIC 中断源对应一个 Priority 寄存器，用于配置该中断源的优先级。每个寄存器的宽度是 4 字节（32 位）
- 最多 1024 个 中断源，0 号中断源不可用。
- 寄存器映射地址 = $\text{BASE} + (\text{中断源 ID}) * 4$
- QEMU-virt 支持 7 个优先级。0 表示对该中断源禁用中断。其余优先级，1 最低，7 最高。
- **如果两个中断源优先级相同，则根据中断源的 ID 值进一步区分优先级，ID 值越小的优先级越高。**

PLIC 编程接口 - 中断源 Pending 寄存器 (1)

PLIC Memory Map

```
base + 0x000000: Reserved (interrupt source 0 does not exist)
base + 0x000004: Interrupt source 1 priority
base + 0x000008: Interrupt source 2 priority
...
base + 0x000FFC: Interrupt source 1023 priority
base + 0x001000: Interrupt Pending bit 0-31
base + 0x00107C: Interrupt Pending bit 992-1023
...
base + 0x002000: Enable bits for sources 0-31 on context 0
base + 0x002004: Enable bits for sources 32-63 on context 0
...
base + 0x00207C: Enable bits for sources 992-1023 on context 0
base + 0x002080: Enable bits for sources 0-31 on context 1
base + 0x002084: Enable bits for sources 32-63 on context 1
...
base + 0x0020FC: Enable bits for sources 992-1023 on context 1
base + 0x002100: Enable bits for sources 0-31 on context 2
base + 0x002104: Enable bits for sources 32-63 on context 2
...
base + 0x00217C: Enable bits for sources 992-1023 on context 2
...
base + 0x1F1F80: Enable bits for sources 0-31 on context 15871
base + 0x1F1F84: Enable bits for sources 32-63 on context 15871
base + 0x1F1FFC: Enable bits for sources 992-1023 on context 15871
...
base + 0x1FFFFC: Reserved
base + 0x200000: Priority threshold for context 0
base + 0x200004: Claim/complete for context 0
base + 0x200008: Reserved
...
base + 0x200FFC: Reserved
base + 0x201000: Priority threshold for context 1
base + 0x201004: Claim/complete for context 1
...
base + 0x3FFF000: Priority threshold for context 15871
base + 0x3FFF004: Claim/complete for context 15871
base + 0x3FFF008: Reserved
...
base + 0x3FFFFFFC: Reserved
```

- 每一个 bit 对应一个中断源，如果为 1 表示该中断源上发生了中断（进入 Pending 状态），有待 hart 处理，否则表示该中断源上当前无中断发生。
- 按 32 位字（4 字节）访问
- 寄存器映射地址 = $\text{BASE} + 0x1000 + (\text{中断源ID} / 32) * 4$
- Pending 寄存器中断的 Pending 状态可以通过 claim 方式清除。
- 第一个 Pending 寄存器的第 0 位对应不存在的 0 号中断源，其值永远为 0。

PLIC 编程接口 - 中断源 Pending 寄存器 (2)

PLIC Memory Map

```
base + 0x000000: Reserved (interrupt source 0 does not exist)
base + 0x000004: Interrupt source 1 priority
base + 0x000008: Interrupt source 2 priority
...
base + 0x000FFC: Interrupt source 1023 priority
base + 0x001000: Interrupt Pending bit 0-31
base + 0x00107C: Interrupt Pending bit 992-1023
...
base + 0x002000: Enable bits for sources 0-31 on context 0
base + 0x002004: Enable bits for sources 32-63 on context 0
...
base + 0x00207C: Enable bits for sources 992-1023 on context 0
base + 0x002080: Enable bits for sources 0-31 on context 1
base + 0x002084: Enable bits for sources 32-63 on context 1
...
base + 0x0020FC: Enable bits for sources 992-1023 on context 1
base + 0x002100: Enable bits for sources 0-31 on context 2
base + 0x002104: Enable bits for sources 32-63 on context 2
...
base + 0x00217C: Enable bits for sources 992-1023 on context 2
...
base + 0x1F1F80: Enable bits for sources 0-31 on context 15871
base + 0x1F1F84: Enable bits for sources 32-63 on context 15871
base + 0x1F1FFC: Enable bits for sources 992-1023 on context 15871
...
base + 0x1FFFFC: Reserved
base + 0x200000: Priority threshold for context 0
base + 0x200004: Claim/complete for context 0
base + 0x200008: Reserved
...
base + 0x200FFC: Reserved
base + 0x201000: Priority threshold for context 1
base + 0x201004: Claim/complete for context 1
...
base + 0x3FFF00: Priority threshold for context 15871
base + 0x3FFF04: Claim/complete for context 15871
base + 0x3FFF08: Reserved
...
base + 0x3FFFFFFC: Reserved
```

- 每一个 bit 对应一个中断源，如果为 1 表示该中断源上发生了中断（进入 Pending 状态），有待 hart 处理，否则表示该中断源上当前无中断发生。
- **按 32 位字 (4 字节) 访问**
- 寄存器映射地址 = $\text{BASE} + 0x1000 + (\text{中断源ID} / 32) * 4$
- Pending 寄存器中断的 Pending 状态可以通过 claim 方式清除。
- 第一个 Pending 寄存器的第 0 位对应不存在的 0 号中断源，其值永远为 0。

PLIC 编程接口 - 中断源 Pending 寄存器 (3)

PLIC Memory Map

```
base + 0x000000: Reserved (interrupt source 0 does not exist)
base + 0x000004: Interrupt source 1 priority
base + 0x000008: Interrupt source 2 priority
...
base + 0x000FFC: Interrupt source 1023 priority
base + 0x001000: Interrupt Pending bit 0-31
base + 0x00107C: Interrupt Pending bit 992-1023
...
base + 0x002000: Enable bits for sources 0-31 on context 0
base + 0x002004: Enable bits for sources 32-63 on context 0
...
base + 0x00207C: Enable bits for sources 992-1023 on context 0
base + 0x002080: Enable bits for sources 0-31 on context 1
base + 0x002084: Enable bits for sources 32-63 on context 1
...
base + 0x0020FC: Enable bits for sources 992-1023 on context 1
base + 0x002100: Enable bits for sources 0-31 on context 2
base + 0x002104: Enable bits for sources 32-63 on context 2
...
base + 0x00217C: Enable bits for sources 992-1023 on context 2
...
base + 0x1F1F80: Enable bits for sources 0-31 on context 15871
base + 0x1F1F84: Enable bits for sources 32-63 on context 15871
base + 0x1F1FFC: Enable bits for sources 992-1023 on context 15871
...
base + 0x1FFFC: Reserved
base + 0x200000: Priority threshold for context 0
base + 0x200004: Claim/complete for context 0
base + 0x200008: Reserved
...
base + 0x200FFC: Reserved
base + 0x201000: Priority threshold for context 1
base + 0x201004: Claim/complete for context 1
...
base + 0x3FFF00: Priority threshold for context 15871
base + 0x3FFF04: Claim/complete for context 15871
base + 0x3FFF08: Reserved
...
base + 0x3FFFFFC: Reserved
```

- 每一个 bit 对应一个中断源，如果为 1 表示该中断源上发生了中断（进入 Pending 状态），有待 hart 处理，否则表示该中断源上当前无中断发生。
- 按 32 位字（4 字节）访问
- 寄存器映射地址 = $\text{BASE} + 0x1000 + (\text{中断源ID} / 32) * 4$
- Pending 寄存器中断的 Pending 状态可以通过 claim 方式清除。
- 第一个 Pending 寄存器的第 0 位对应不存在的 0 号中断源，其值永远为 0。

PLIC 编程接口 - 中断源 Pending 寄存器 (4)

PLIC Memory Map

```
base + 0x000000: Reserved (interrupt source 0 does not exist)
base + 0x000004: Interrupt source 1 priority
base + 0x000008: Interrupt source 2 priority
...
base + 0x000FFC: Interrupt source 1023 priority
base + 0x001000: Interrupt Pending bit 0-31
base + 0x00107C: Interrupt Pending bit 992-1023
...
base + 0x002000: Enable bits for sources 0-31 on context 0
base + 0x002004: Enable bits for sources 32-63 on context 0
...
base + 0x00207C: Enable bits for sources 992-1023 on context 0
base + 0x002080: Enable bits for sources 0-31 on context 1
base + 0x002084: Enable bits for sources 32-63 on context 1
...
base + 0x0020FC: Enable bits for sources 992-1023 on context 1
base + 0x002100: Enable bits for sources 0-31 on context 2
base + 0x002104: Enable bits for sources 32-63 on context 2
...
base + 0x00217C: Enable bits for sources 992-1023 on context 2
...
base + 0x1F1F80: Enable bits for sources 0-31 on context 15871
base + 0x1F1F84: Enable bits for sources 32-63 on context 15871
base + 0x1F1FFC: Enable bits for sources 992-1023 on context 15871
...
base + 0x1FFFFC: Reserved
base + 0x200000: Priority threshold for context 0
base + 0x200004: Claim/complete for context 0
base + 0x200008: Reserved
...
base + 0x200FFC: Reserved
base + 0x201000: Priority threshold for context 1
base + 0x201004: Claim/complete for context 1
...
base + 0x3FFF00: Priority threshold for context 15871
base + 0x3FFF04: Claim/complete for context 15871
base + 0x3FFF08: Reserved
...
base + 0x3FFFFFFC: Reserved
```

- 每一个 bit 对应一个中断源，如果为 1 表示该中断源上发生了中断（进入 Pending 状态），有待 hart 处理，否则表示该中断源上当前无中断发生。
- 按 32 位字（4 字节）访问
- 寄存器映射地址 = $\text{BASE} + 0x1000 + (\text{中断源ID} / 32) * 4$
- **Pending 寄存器中断的 Pending 状态可以通过 claim 方式清除。**
- 第一个 Pending 寄存器的第 0 位对应不存在的 0 号中断源，其值永远为 0。

PLIC 编程接口 - 中断源 Pending 寄存器 (5)

PLIC Memory Map

```
base + 0x000000: Reserved (interrupt source 0 does not exist)
base + 0x000004: Interrupt source 1 priority
base + 0x000008: Interrupt source 2 priority
...
base + 0x000FFC: Interrupt source 1023 priority
base + 0x001000: Interrupt Pending bit 0-31
base + 0x00107C: Interrupt Pending bit 992-1023
...
base + 0x002000: Enable bits for sources 0-31 on context 0
base + 0x002004: Enable bits for sources 32-63 on context 0
...
base + 0x00207C: Enable bits for sources 992-1023 on context 0
base + 0x002080: Enable bits for sources 0-31 on context 1
base + 0x002084: Enable bits for sources 32-63 on context 1
...
base + 0x0020FC: Enable bits for sources 992-1023 on context 1
base + 0x002100: Enable bits for sources 0-31 on context 2
base + 0x002104: Enable bits for sources 32-63 on context 2
...
base + 0x00217C: Enable bits for sources 992-1023 on context 2
...
base + 0x1F1F80: Enable bits for sources 0-31 on context 15871
base + 0x1F1F84: Enable bits for sources 32-63 on context 15871
base + 0x1F1FFC: Enable bits for sources 992-1023 on context 15871
...
base + 0x1FFFFC: Reserved
base + 0x200000: Priority threshold for context 0
base + 0x200004: Claim/complete for context 0
base + 0x200008: Reserved
...
base + 0x200FFC: Reserved
base + 0x201000: Priority threshold for context 1
base + 0x201004: Claim/complete for context 1
...
base + 0x3FFF000: Priority threshold for context 15871
base + 0x3FFF004: Claim/complete for context 15871
base + 0x3FFF008: Reserved
...
base + 0x3FFFFFFC: Reserved
```

- 每一个 bit 对应一个中断源，如果为 1 表示该中断源上发生了中断（进入 Pending 状态），有待 hart 处理，否则表示该中断源上当前无中断发生。
- 按 32 位字（4 字节）访问
- 寄存器映射地址 = $\text{BASE} + 0x1000 + (\text{中断源ID} / 32) * 4$
- Pending 寄存器中断的 Pending 状态可以通过 claim 方式清除。
- **第一个 Pending 寄存器的第 0 位对应不存在的 0 号中断源，其值永远为 0。**

PLIC 编程接口 - 中断源 Enable 寄存器 (1)

PLIC Memory Map

```
base + 0x000000: Reserved (interrupt source 0 does not exist)
base + 0x000004: Interrupt source 1 priority
base + 0x000008: Interrupt source 2 priority
...
base + 0x000FFC: Interrupt source 1023 priority
base + 0x001000: Interrupt Pending bit 0-31
base + 0x00107C: Interrupt Pending bit 992-1023
...
base + 0x002000: Enable bits for sources 0-31 on context 0
base + 0x002004: Enable bits for sources 32-63 on context 0
...
base + 0x00207C: Enable bits for sources 992-1023 on context 0
base + 0x002080: Enable bits for sources 0-31 on context 1
base + 0x002084: Enable bits for sources 32-63 on context 1
...
base + 0x0020FC: Enable bits for sources 992-1023 on context 1
base + 0x002100: Enable bits for sources 0-31 on context 2
base + 0x002104: Enable bits for sources 32-63 on context 2
...
base + 0x00217C: Enable bits for sources 992-1023 on context 2
...
base + 0x1F1F80: Enable bits for sources 0-31 on context 15871
base + 0x1F1F84: Enable bits for sources 32-63 on context 15871
base + 0x1F1FFC: Enable bits for sources 992-1023 on context 15871
...
base + 0x1FFFFC: Reserved
base + 0x200000: Priority threshold for context 0
base + 0x200004: Claim/complete for context 0
base + 0x200008: Reserved
...
base + 0x200FFC: Reserved
base + 0x201000: Priority threshold for context 1
base + 0x201004: Claim/complete for context 1
...
base + 0x3FFF00: Priority threshold for context 15871
base + 0x3FFF04: Claim/complete for context 15871
base + 0x3FFF08: Reserved
...
base + 0x3FFFFC: Reserved
```

- 按照上下文 (Context) 分成独立的 Enable 寄存器组，每组大小: 0x80 (128 字节)。
- 上下文基地址: 0x2000
- 通常一个上下文对应一种特权模式 (如 M-mode 和 S-mode)。在 QEMU 的 virt 机器中，默认配置下，每个 HART 有两个上下文：
 - 上下文 0: 对应 M-mode
 - 上下文 1: 对应 S-mode
- S-mode 寄存器映射地址 = $\text{BASE} + 0x2000 + (\text{hart}) * 0x100 + 0x80$
- 每个寄存器 (32 位) 的一个 bit 对应一路中断源。将对应的 bit 位设置为 1 表示使能该中断源，否则表示关闭该中断源。

PLIC 编程接口 - 中断源 Enable 寄存器 (2)

PLIC Memory Map

```
base + 0x000000: Reserved (interrupt source 0 does not exist)
base + 0x000004: Interrupt source 1 priority
base + 0x000008: Interrupt source 2 priority
...
base + 0x000FFC: Interrupt source 1023 priority
base + 0x001000: Interrupt Pending bit 0-31
base + 0x00107C: Interrupt Pending bit 992-1023
...
base + 0x002000: Enable bits for sources 0-31 on context 0
base + 0x002004: Enable bits for sources 32-63 on context 0
...
base + 0x00207C: Enable bits for sources 992-1023 on context 0
base + 0x002080: Enable bits for sources 0-31 on context 1
base + 0x002084: Enable bits for sources 32-63 on context 1
...
base + 0x0020FC: Enable bits for sources 992-1023 on context 1
base + 0x002100: Enable bits for sources 0-31 on context 2
base + 0x002104: Enable bits for sources 32-63 on context 2
...
base + 0x00217C: Enable bits for sources 992-1023 on context 2
...
base + 0x1F1F80: Enable bits for sources 0-31 on context 15871
base + 0x1F1F84: Enable bits for sources 32-63 on context 15871
base + 0x1F1FFC: Enable bits for sources 992-1023 on context 15871
...
base + 0x1FFFFC: Reserved
base + 0x200000: Priority threshold for context 0
base + 0x200004: Claim/complete for context 0
base + 0x200008: Reserved
...
base + 0x200FFC: Reserved
base + 0x201000: Priority threshold for context 1
base + 0x201004: Claim/complete for context 1
...
base + 0x3FFF00: Priority threshold for context 15871
base + 0x3FFF04: Claim/complete for context 15871
base + 0x3FFF08: Reserved
...
base + 0x3FFFFFC: Reserved
```

- 按照上下文 (Context) 分成独立的 Enable 寄存器组，每组大小: 0x80 (128 字节)。
- 上下文基地址: 0x2000
- 通常一个上下文对应一种特权模式 (如 **M-mode** 和 **S-mode**)。在 QEMU 的 virt 机器中，默认配置下，每个 HART 有两个上下文：
 - 上下文 0: 对应 **M-mode**
 - 上下文 1: 对应 **S-mode**
- **S-mode 寄存器映射地址 = BASE + 0x2000 + (hart) * 0x100 + 0x80**

```
#define PLIC_SENABLE(hart) (PLIC + 0x2080 + (hart)*0x100)
```

应的 bit 位设置为 1 表示使能该中断源，否则表示关闭该中断源。

PLIC 编程接口 - 中断源 Enable 寄存器 (3)

PLIC Memory Map

```
base + 0x000000: Reserved (interrupt source 0 does not exist)
base + 0x000004: Interrupt source 1 priority
base + 0x000008: Interrupt source 2 priority
...
base + 0x000FFC: Interrupt source 1023 priority
base + 0x001000: Interrupt Pending bit 0-31
base + 0x00107C: Interrupt Pending bit 992-1023
...
```

```
base + 0x002000: Enable bits for sources 0-31 on context 0
base + 0x002004: Enable bits for sources 32-63 on context 0
...
base + 0x00207C: Enable bits for sources 992-1023 on context 0
base + 0x002080: Enable bits for sources 0-31 on context 1
base + 0x002084: Enable bits for sources 32-63 on context 1
...
base + 0x0020FC: Enable bits for sources 992-1023 on context 1
base + 0x002100: Enable bits for sources 0-31 on context 2
base + 0x002104: Enable bits for sources 32-63 on context 2
...
base + 0x00217C: Enable bits for sources 992-1023 on context 2
...
base + 0x1F1F80: Enable bits for sources 0-31 on context 15871
base + 0x1F1F84: Enable bits for sources 32-63 on context 15871
base + 0x1F1FFC: Enable bits for sources 992-1023 on context 15871
...
```

```
base + 0x1FFFC: Reserved
base + 0x200000: Priority threshold for context 0
base + 0x200004: Claim/complete for context 0
base + 0x200008: Reserved
...
base + 0x200FFC: Reserved
base + 0x201000: Priority threshold for context 1
base + 0x201004: Claim/complete for context 1
...
base + 0x3FFF00: Priority threshold for context 15871
base + 0x3FFF04: Claim/complete for context 15871
base + 0x3FFF08: Reserved
...
base + 0x3FFFFFC: Reserved
```

- 按照上下文 (Context) 分成独立的 Enable 寄存器组，每组大小: 0x80 (128 字节)。

- 上下文基地址: 0x2000

```
void plicinithart(void)
{
    int hart = cpuid();
    // set enable bits for this hart's S-mode
    // for the uart and virtio disk.
    *(uint32*)PLIC_SENABLE(hart) = (1 << UART0_IRQ);
    // .....
}
```

- 每个寄存器 (32 位) 的一个 bit 对应一路中断源。将对应的 bit 位设置为 1 表示使能该中断源，否则表示关闭该中断源。

S-
每个

)*

PLIC 编程接口 - 中断源 Threshold 寄存器 (1)

PLIC Memory Map

```
base + 0x000000: Reserved (interrupt source 0 does not exist)
base + 0x000004: Interrupt source 1 priority
base + 0x000008: Interrupt source 2 priority
...
base + 0x000FFC: Interrupt source 1023 priority
base + 0x001000: Interrupt Pending bit 0-31
base + 0x00107C: Interrupt Pending bit 992-1023
...
base + 0x002000: Enable bits for sources 0-31 on context 0
base + 0x002004: Enable bits for sources 32-63 on context 0
...
base + 0x00207C: Enable bits for sources 992-1023 on context 0
base + 0x002080: Enable bits for sources 0-31 on context 1
base + 0x002084: Enable bits for sources 32-63 on context 1
...
base + 0x0020FC: Enable bits for sources 992-1023 on context 1
base + 0x002100: Enable bits for sources 0-31 on context 2
base + 0x002104: Enable bits for sources 32-63 on context 2
...
base + 0x00217C: Enable bits for sources 992-1023 on context 2
...
base + 0x1F1F80: Enable bits for sources 0-31 on context 15871
base + 0x1F1F84: Enable bits for sources 32-63 on context 15871
base + 0x1F1FFC: Enable bits for sources 992-1023 on context 15871
...
base + 0x1FFFFC: Reserved
base + 0x200000: Priority threshold for context 0
base + 0x200004: Claim/complete for context 0
base + 0x200008: Reserved
...
base + 0x200FFC: Reserved
base + 0x201000: Priority threshold for context 1
base + 0x201004: Claim/complete for context 1
...
base + 0x3FFF00: Priority threshold for context 15871
base + 0x3FFF04: Claim/complete for context 15871
base + 0x3FFF08: Reserved
...
base + 0x3FFFFC: Reserved
```

- 按照上下文 (Context) 分组, 每个上下文寄存器组的大小为 **0x1000 (4KB)**。
- 上下文基地址: **0x200000**
- 每组含有一个 Threshold 寄存器, 寄存器大小: 4 字节。
- 类似地, 在 QEMU 的 virt 机器中, 默认配置下, 每个 HART 有两个上下文:
 - 上下文 0: 对应 M-mode
 - 上下文 1: 对应 S-mode
- S-mode 寄存器映射地址 = $\text{BASE} + 0x200000 + (\text{hart}) * 0x2000 + 0x1000$
- Threshold 寄存器用于设置中断优先级的阈值。所有小于或者等于 ($\leq$) 该阈值的中断源即使发生了也会被 PLIC 丢弃。特别地, 当阈值为 0 时允许所有中断源上发生的中断; 当阈值为 7 时丢弃所有中断源上发生的中断。

PLIC 编程接口 - 中断源 Threshold 寄存器 (2)

PLIC Memory Map

```
base + 0x000000: Reserved (interrupt source 0 does not exist)
base + 0x000004: Interrupt source 1 priority
base + 0x000008: Interrupt source 2 priority
...
base + 0x000FFC: Interrupt source 1023 priority
base + 0x001000: Interrupt Pending bit 0-31
base + 0x00107C: Interrupt Pending bit 992-1023
...
base + 0x002000: Enable bits for sources 0-31 on context 0
base + 0x002004: Enable bits for sources 32-63 on context 0
...
base + 0x00207C: Enable bits for sources 992-1023 on context 0
base + 0x002080: Enable bits for sources 0-31 on context 1
base + 0x002084: Enable bits for sources 32-63 on context 1
...
base + 0x0020FC: Enable bits for sources 992-1023 on context 1
base + 0x002100: Enable bits for sources 0-31 on context 2
base + 0x002104: Enable bits for sources 32-63 on context 2
...
base + 0x00217C: Enable bits for sources 992-1023 on context 2
...
base + 0x1F1F80: Enable bits for sources 0-31 on context 15871
base + 0x1F1F84: Enable bits for sources 32-63 on context 15871
base + 0x1F1FFC: Enable bits for sources 992-1023 on context 15871
...
base + 0x1FFFFC: Reserved
base + 0x200000: Priority threshold for context 0
base + 0x200004: Claim/complete for context 0
base + 0x200008: Reserved
...
base + 0x200FFC: Reserved
base + 0x201000: Priority threshold for context 1
base + 0x201004: Claim/complete for context 1
...
base + 0x3FFF00: Priority threshold for context 15871
base + 0x3FFF04: Claim/complete for context 15871
base + 0x3FFF08: Reserved
...
base + 0x3FFFFC: Reserved
```

- 按照上下文 (Context) 分组, 每个上下文寄存器组的大小为 0x1000 (4KB)。
- 上下文基地址: 0x200000
- 每组含有一个 Threshold 寄存器, 寄存器大小: 4 字节。**
- 类似地, 在 QEMU 的 virt 机器中, 默认配置下, 每个 HART 有两个上下文:
 - 上下文 0: 对应 M-mode
 - 上下文 1: 对应 S-mode
- S-mode 寄存器映射地址 = $\text{BASE} + 0x200000 + (\text{hart}) * 0x2000 + 0x1000$
- Threshold 寄存器用于设置中断优先级的阈值。所有小于或者等于 ($\leq$) 该阈值的中断源即使发生了也会被 PLIC 丢弃。特别地, 当阈值为 0 时允许所有中断源上发生的中断; 当阈值为 7 时丢弃所有中断源上发生的中断。

PLIC 编程接口 - 中断源 Threshold 寄存器 (3)

PLIC Memory Map

```
base + 0x000000: Reserved (interrupt source 0 does not exist)
base + 0x000004: Interrupt source 1 priority
base + 0x000008: Interrupt source 2 priority
...
base + 0x000FFC: Interrupt source 1023 priority
base + 0x001000: Interrupt Pending bit 0-31
base + 0x00107C: Interrupt Pending bit 992-1023
...
base + 0x002000: Enable bits for sources 0-31 on context 0
base + 0x002004: Enable bits for sources 32-63 on context 0
...
base + 0x00207C: Enable bits for sources 992-1023 on context 0
base + 0x002080: Enable bits for sources 0-31 on context 1
base + 0x002084: Enable bits for sources 32-63 on context 1
...
base + 0x0020FC: Enable bits for sources 992-1023 on context 1
base + 0x002100: Enable bits for sources 0-31 on context 2
base + 0x002104: Enable bits for sources 32-63 on context 2
...
base + 0x00217C: Enable bits for sources 992-1023 on context 2
...
base + 0x1F1F80: Enable bits for sources 0-31 on context 15871
base + 0x1F1F84: Enable bits for sources 32-63 on context 15871
base + 0x1F1FFC: Enable bits for sources 992-1023 on context 15871
...
base + 0x1FFFFC: Reserved
base + 0x200000: Priority threshold for context 0
base + 0x200004: Claim/complete for context 0
base + 0x200008: Reserved
...
base + 0x200FFC: Reserved
base + 0x201000: Priority threshold for context 1
base + 0x201004: Claim/complete for context 1
...
base + 0x3FFF000: Priority threshold for context 15871
base + 0x3FFF004: Claim/complete for context 15871
base + 0x3FFF008: Reserved
...
base + 0x3FFFFFFC: Reserved
```

- 按照上下文 (Context) 分组, 每个上下文寄存器组的大小为 0x1000 (4KB)。
- 上下文基地址: 0x200000
- 每组含有一个 Threshold 寄存器, 寄存器大小: 4 字节。
- 类似地, 在 QEMU 的 virt 机器中, 默认配置下, 每个 HART 有两个上下文:
 - 上下文 0: 对应 M-mode
 - 上下文 1: 对应 S-mode
- S-mode 寄存器映射地址 = $\text{BASE} + 0x200000 + (\text{hart}) * 0x2000 + 0x1000$

```
#define PLIC_SPRIORITY(hart) (PLIC + 0x201000 + (hart)*0x2000)
```

或者等于 ($\leq$) 该阈值的中断源即使发生了也会被 PLIC 丢弃。特别地, 当阈值为 0 时允许所有中断源上发生的中断; 当阈值为 7 时丢弃所有中断源上发生的中断。

PLIC 编程接口 - 中断源 Threshold 寄存器 (4)

PLIC Memory Map

```
base + 0x000000: Reserved (interrupt source 0 does not exist)
base + 0x000004: Interrupt source 1 priority
base + 0x000008: Interrupt source 2 priority
...
base + 0x000FFC: Interrupt source 1023 priority
base + 0x001000: Interrupt Pending bit 0-31
base + 0x00107C: Interrupt Pending bit 992-1023
...
base + 0x002000: Enable bits for sources 0-31 on context 0
base + 0x002004: Enable bits for sources 32-63 on context 0
...
base + 0x00207C: Enable bits for sources 992-1023 on context 0
base + 0x002080: Enable bits for sources 0-31 on context 1
base + 0x002084: Enable bits for sources 32-63 on context 1
...
base + 0x0020FC: Enable bits for sources 992-1023 on context 1
base + 0x002100: Enable bits for sources 0-31 on context 2
base + 0x002104: Enable bits for sources 32-63 on context 2
...
base + 0x00217C: Enable bits for sources 992-1023 on context 2
...
base + 0x1F1F80: Enable bits for sources 0-31 on context 15871
base + 0x1F1F84: Enable bits for sources 32-63 on context 15871
base + 0x1F1FFC: Enable bits for sources 992-1023 on context 15871
...
base + 0x1FFFFC: Reserved
base + 0x200000: Priority threshold for context 0
base + 0x200004: Claim/complete for context 0
base + 0x200008: Reserved
...
base + 0x200FFC: Reserved
base + 0x201000: Priority threshold for context 1
base + 0x201004: Claim/complete for context 1
...
base + 0x3FFF00: Priority threshold for context 15871
base + 0x3FFF04: Claim/complete for context 15871
base + 0x3FFF08: Reserved
...
base + 0x3FFFFC: Reserved
```

- 按照上下文 (Context) 分组, 每个上下文寄存器组的大小为 0x1000 (4KB)。
- 上下文基地址: 0x200000

```
void plicinithart(void)
{
    int hart = cpuid();
    // .....
    // set this hart's S-mode priority threshold to 0.
    *(uint32*)PLIC_SPRIORITY(hart) = 0;
}
```

- Threshold 寄存器用于设置中断优先级的阈值。所有小于或者等于 ($\leq$) 该阈值的中断源即使发生了也会被 PLIC 丢弃。特别地, 当阈值为 0 时允许所有中断源上发生的中断; 当阈值为 7 时丢弃所有中断源上发生的中断。**

PLIC 编程接口 - 中断源 Claim/Complete 寄存器 (1)

PLIC Memory Map

```
base + 0x000000: Reserved (interrupt source 0 does not exist)
base + 0x000004: Interrupt source 1 priority
base + 0x000008: Interrupt source 2 priority
...
base + 0x000FFC: Interrupt source 1023 priority
base + 0x001000: Interrupt Pending bit 0-31
base + 0x00107C: Interrupt Pending bit 992-1023
...
base + 0x002000: Enable bits for sources 0-31 on context 0
base + 0x002004: Enable bits for sources 32-63 on context 0
...
base + 0x00207C: Enable bits for sources 992-1023 on context 0
base + 0x002080: Enable bits for sources 0-31 on context 1
base + 0x002084: Enable bits for sources 32-63 on context 1
...
base + 0x0020FC: Enable bits for sources 992-1023 on context 1
base + 0x002100: Enable bits for sources 0-31 on context 2
base + 0x002104: Enable bits for sources 32-63 on context 2
...
base + 0x00217C: Enable bits for sources 992-1023 on context 2
...
base + 0x1F1F80: Enable bits for sources 0-31 on context 15871
base + 0x1F1F84: Enable bits for sources 32-63 on context 15871
base + 0x1F1FFC: Enable bits for sources 992-1023 on context 15871
...
base + 0x1FFFFC: Reserved
base + 0x200000: Priority threshold for context 0
base + 0x200004: Claim/complete for context 0
base + 0x200008: Reserved
...
base + 0x200FFC: Reserved
base + 0x201000: Priority threshold for context 1
base + 0x201004: Claim/complete for context 1
...
base + 0x3FFF000: Priority threshold for context 15871
base + 0x3FFF004: Claim/complete for context 15871
base + 0x3FFF008: Reserved
...
base + 0x3FFFFFFC: Reserved
```

- 和 Threshold 一起按照上下文 (Context) 分组, 每个上下文寄存器组的大小为 0x1000 (4KB)。
- 上下文基地址: 0x200000
- Claim 和 Complete 是同一个寄存器, 每组含有一个, 即每个 Hart 一个。寄存器大小: 4 字节。
- 类似地, 在 QEMU 的 virt 机器中, 默认配置下, 每个 HART 有两个上下文:
 - 上下文 0: 对应 M-mode
 - 上下文 1: 对应 S-mode
- S-mode 寄存器映射地址 = $\text{BASE} + 0x20000 + (\text{hart}) * 0x2000 + 0x1000 + 4$
- 对该寄存器执行读操作称之为 Claim, 即获取当前发生的最高优先级的中断源 ID。Claim 成功后会清除对应的 Pending 位。
- 对该寄存器执行写操作称之为 Complete。所谓 Complete 指的是通知 PLIC 对该路中断的处理已经结束。

PLIC 编程接口 - 中断源 Claim/Complete 寄存器 (2)

PLIC Memory Map

```
base + 0x000000: Reserved (interrupt source 0 does not exist)
base + 0x000004: Interrupt source 1 priority
base + 0x000008: Interrupt source 2 priority
...
base + 0x000FFC: Interrupt source 1023 priority
base + 0x001000: Interrupt Pending bit 0-31
base + 0x00107C: Interrupt Pending bit 992-1023
...
base + 0x002000: Enable bits for sources 0-31 on context 0
base + 0x002004: Enable bits for sources 32-63 on context 0
...
base + 0x00207C: Enable bits for sources 992-1023 on context 0
base + 0x002080: Enable bits for sources 0-31 on context 1
base + 0x002084: Enable bits for sources 32-63 on context 1
...
base + 0x0020FC: Enable bits for sources 992-1023 on context 1
base + 0x002100: Enable bits for sources 0-31 on context 2
base + 0x002104: Enable bits for sources 32-63 on context 2
...
base + 0x00217C: Enable bits for sources 992-1023 on context 2
...
base + 0x1F1F80: Enable bits for sources 0-31 on context 15871
base + 0x1F1F84: Enable bits for sources 32-63 on context 15871
base + 0x1F1FFC: Enable bits for sources 992-1023 on context 15871
...
base + 0x1FFFFC: Reserved
base + 0x200000: Priority threshold for context 0
base + 0x200004: Claim/complete for context 0
base + 0x200008: Reserved
...
base + 0x200FFC: Reserved
base + 0x201000: Priority threshold for context 1
base + 0x201004: Claim/complete for context 1
...
base + 0x3FFF000: Priority threshold for context 15871
base + 0x3FFF004: Claim/complete for context 15871
base + 0x3FFF008: Reserved
...
base + 0x3FFFFFFC: Reserved
```

- 和 Threshold 一起按照上下文 (Context) 分组, 每个上下文寄存器组的大小为 0x1000 (4KB)。
- 上下文基地址: 0x200000
- Claim 和 Complete 是同一个寄存器, 每组含有一个, 即每个 Hart 一个。寄存器大小: 4 字节。
- 类似地, 在 QEMU 的 virt 机器中, 默认配置下, 每个 HART 有两个上下文:
 - 上下文 0: 对应 M-mode
 - 上下文 1: 对应 S-mode
- S-mode 寄存器映射地址 = $\text{BASE} + 0x20000 + (\text{hart}) * 0x2000 + 0x1000 + 4$

```
#define PLIC_SCLAIM(hart) (PLIC + 0x201004 + (hart)*0x2000)
```

最高优先级的中断源 ID。Claim 成功后会清除对应的 Pending 位。

- 对该寄存器执行写操作称之为 Complete。所谓 Complete 指的是通知 PLIC 对该路中断的处理已经结束。

PLIC 编程接口 - 中断源 Claim/Complete 寄存器 (3)

PLIC Memory Map

```
base + 0x000000: Reserved (interrupt source 0 does not exist)
base + 0x000004: Interrupt source 1 priority
base + 0x000008: Interrupt source 2 priority
...
base + 0x000FFC: Interrupt source 1023 priority
base + 0x001000: Interrupt Pending bit 0-31
base + 0x00107C: Interrupt Pending bit 992-1023
...
base + 0x002000: Enable bits for sources 0-31 on context 0
base + 0x002004: Enable bits for sources 32-63 on context 0
...
base + 0x00207C: Enable bits for sources 992-1023 on context 0
base + 0x002080: Enable bits for sources 0-31 on context 1
base + 0x002084: Enable bits for sources 32-63 on context 1
...
base + 0x0020FC: Enable bits for sources 992-1023 on context 1
base + 0x002100: Enable bits for sources 0-31 on context 2
base + 0x002104: Enable bits for sources 32-63 on context 2
...
base + 0x00217C: Enable bits for sources 992-1023 on context 2
...
base + 0x1F1F80: Enable bits for sources 0-31 on context 15871
base + 0x1F1F84: Enable bits for sources 32-63 on context 15871
base + 0x1F1FFC: Enable bits for sources 992-1023 on context 15871
...
base + 0x1FFFFC: Reserved
base + 0x200000: Priority threshold for context 0
base + 0x200004: Claim/complete for context 0
base + 0x200008: Reserved
...
base + 0x200FFC: Reserved
base + 0x201000: Priority threshold for context 1
base + 0x201004: Claim/complete for context 1
...
base + 0x3FFF00: Priority threshold for context 15871
base + 0x3FFF04: Claim/complete for context 15871
base + 0x3FFF08: Reserved
...
base + 0x3FFFFFFC: Reserved
```

- 和 Threshold 一起按照上下文 (Context) 分组, 每个上下文寄存器组的大小为 0x1000 (4KB)。
 - 上下文基地址: 0x200000
 - Claim 和 Complete 是同一个寄存器, 每组含有一个, 即
- ```
// ask the PLIC what interrupt we should serve.
int plic_claim(void)
{
 int hart = cpuid();
 int irq = *(uint32*)PLIC_SCLAIM(hart);
 return irq;
}
```
- 对该寄存器执行读操作称之为 **Claim**, 即获取当前发生的最高优先级的中断源 ID。Claim 成功后会清除对应的 **Pending 位**。
  - 对该寄存器执行写操作称之为 Complete。所谓 Complete 指的是通知 PLIC 对该路中断的处理已经结束。

# PLIC 编程接口 - 中断源 Claim/Complete 寄存器 (4)

PLIC Memory Map

```
base + 0x000000: Reserved (interrupt source 0 does not exist)
base + 0x000004: Interrupt source 1 priority
base + 0x000008: Interrupt source 2 priority
...
base + 0x000FFC: Interrupt source 1023 priority
base + 0x001000: Interrupt Pending bit 0-31
base + 0x00107C: Interrupt Pending bit 992-1023
...
base + 0x002000: Enable bits for sources 0-31 on context 0
base + 0x002004: Enable bits for sources 32-63 on context 0
...
base + 0x00207C: Enable bits for sources 992-1023 on context 0
base + 0x002080: Enable bits for sources 0-31 on context 1
base + 0x002084: Enable bits for sources 32-63 on context 1
...
base + 0x0020FC: Enable bits for sources 992-1023 on context 1
base + 0x002100: Enable bits for sources 0-31 on context 2
base + 0x002104: Enable bits for sources 32-63 on context 2
...
base + 0x00217C: Enable bits for sources 992-1023 on context 2
...
base + 0x1F1F80: Enable bits for sources 0-31 on context 15871
base + 0x1F1F84: Enable bits for sources 32-63 on context 15871
base + 0x1F1FFC: Enable bits for sources 992-1023 on context 15871
...
base + 0x1FFFFC: Reserved
base + 0x200000: Priority threshold for context 0
base + 0x200004: Claim/complete for context 0
base + 0x200008: Reserved
...
base + 0x200FFC: Reserved
base + 0x201000: Priority threshold for context 1
base + 0x201004: Claim/complete for context 1
...
base + 0x3FFF00: Priority threshold for context 15871
base + 0x3FFF04: Claim/complete for context 15871
base + 0x3FFF08: Reserved
...
base + 0x3FFFFC: Reserved
```

- 和 Threshold 一起按照上下文 (Context) 分组，每个上下文寄存器组的大小为 0x1000 (4KB)。

- 上下文基地址: 0x200000

- Claim 和 Complete 是同一个寄存器，每组含有一个，即每个 Hart 一个。寄存器大小: 4 字节。

- 类似地，在 QEMU 的 virt 机器中，默认配置下，每个 HART 有两个上下文:

```
// tell the PLIC we've served this IRQ.
void plic_complete(int irq)
```

- S-n
- 0x2
- 对该
- 最高
- Per

```
{
 int hart = cpuid();
 (uint32)PLIC_SCLAIM(hart) = irq;
}
```

- 对该寄存器执行写操作称之为 **Complete**。所谓 **Complete** 指的是通知 PLIC 对该路中断的处理已经结束。

# PLIC 编程接口 - 处理流程

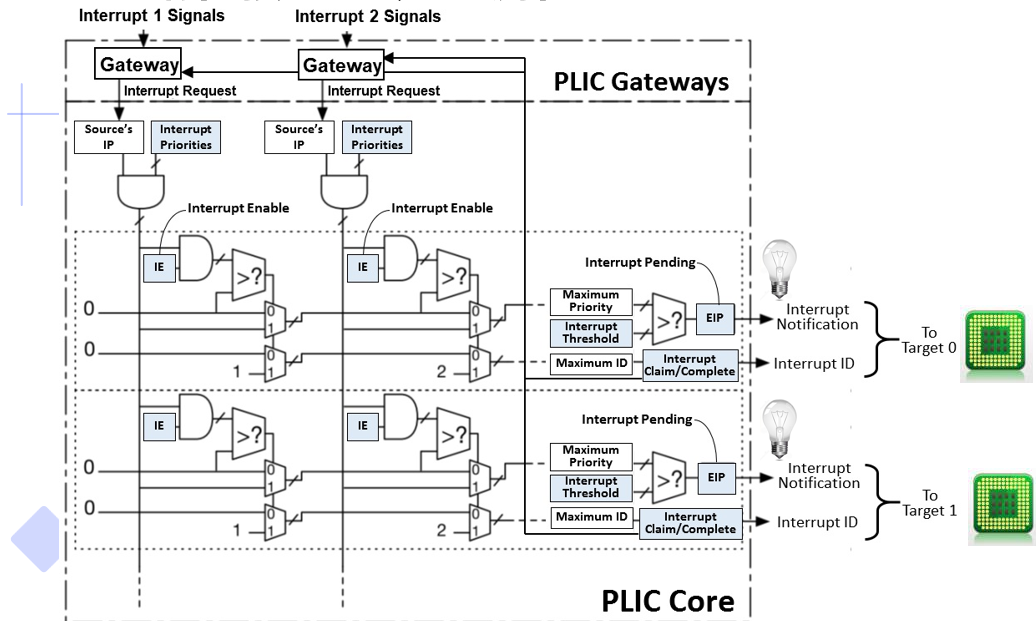


Figure 3. PLIC Operation Parameter Block Diagram

# PLIC 编程接口 - 处理流程

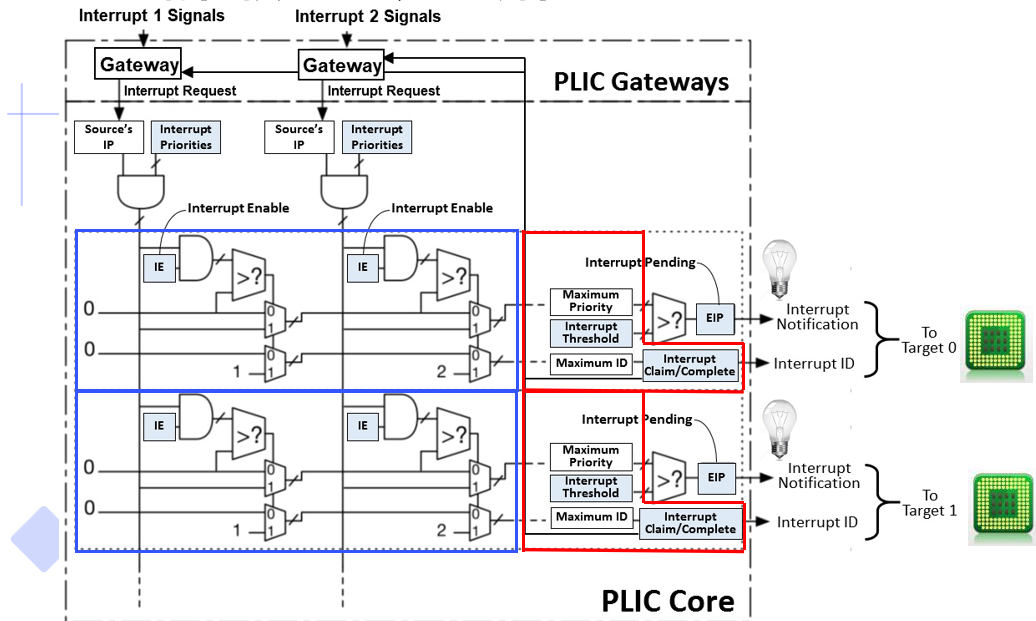


Figure 3. PLIC Operation Parameter Block Diagram



# PLIC 编程接口 - 处理流程

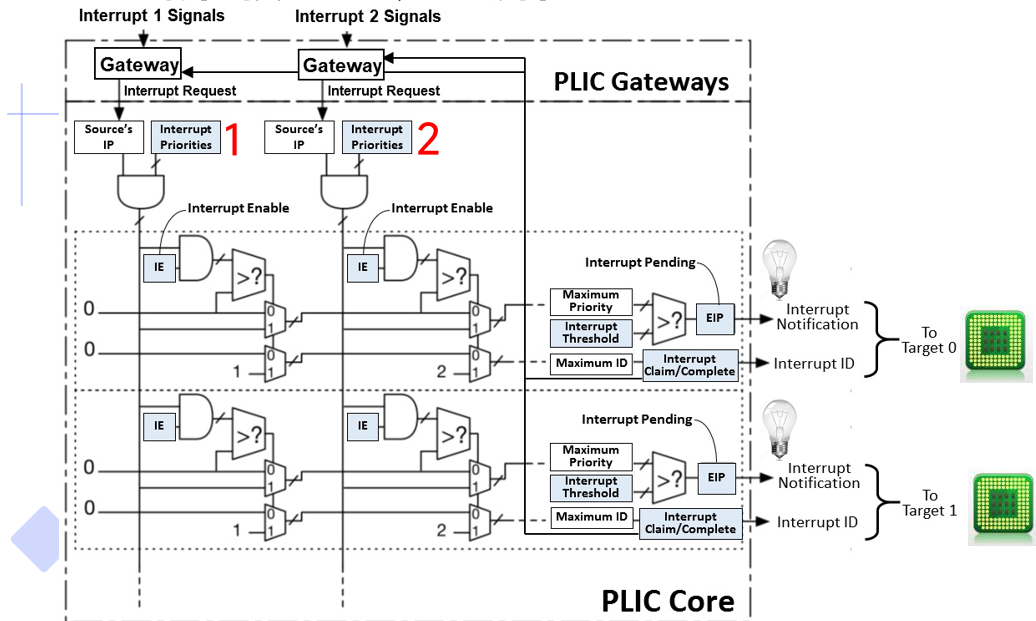


Figure 3. PLIC Operation Parameter Block Diagram

# PLIC 编程接口 - 处理流程

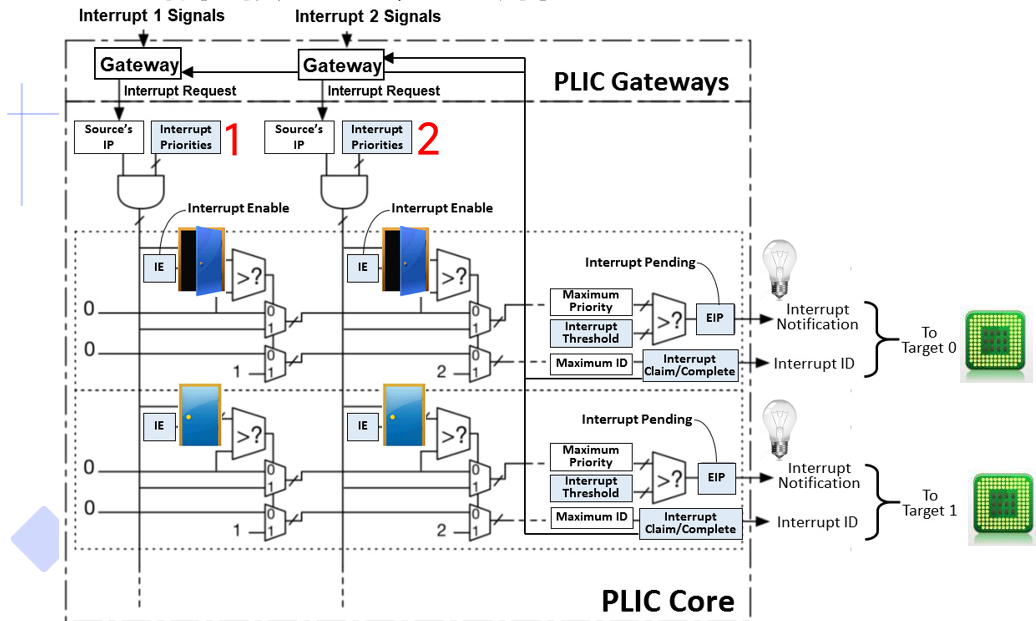


Figure 3. PLIC Operation Parameter Block Diagram

# PLIC 编程接口 - 处理流程

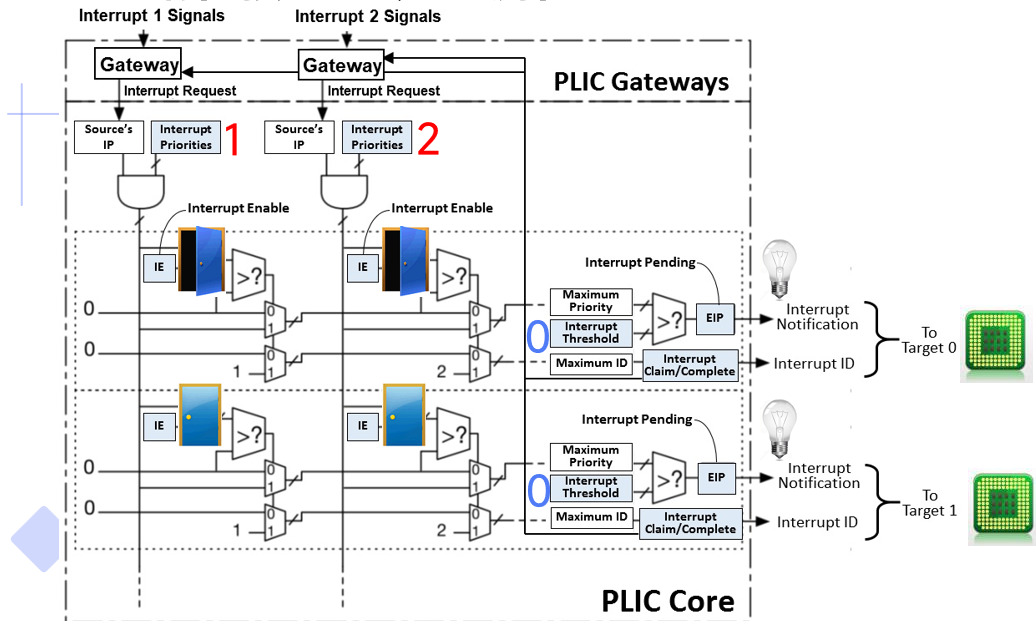


Figure 3. PLIC Operation Parameter Block Diagram

# PLIC 编程接口 - 处理流程

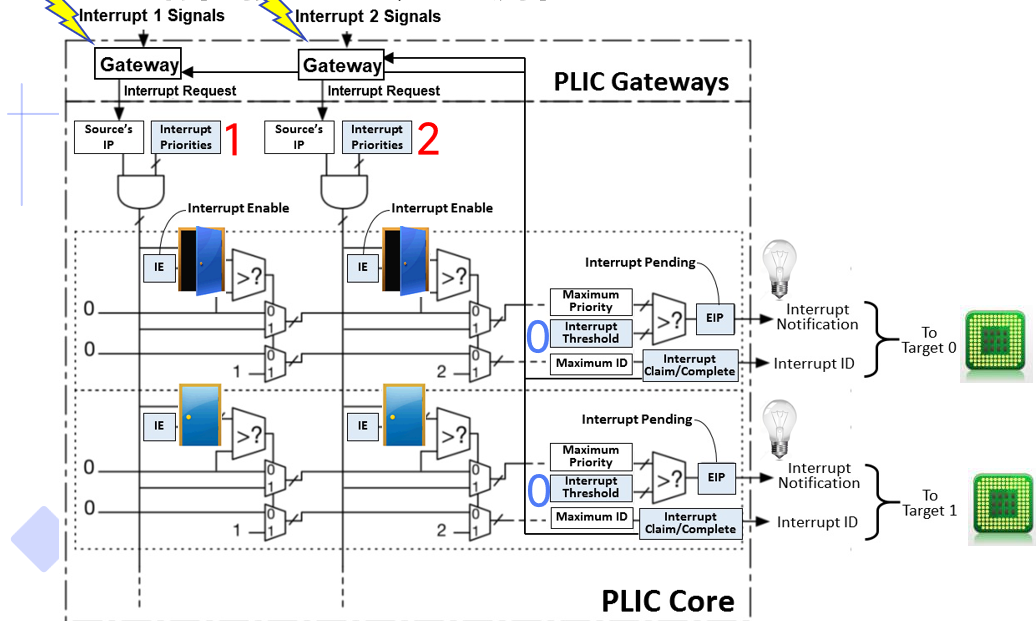


Figure 3. PLIC Operation Parameter Block Diagram

# PLIC 编程接口 - 处理流程

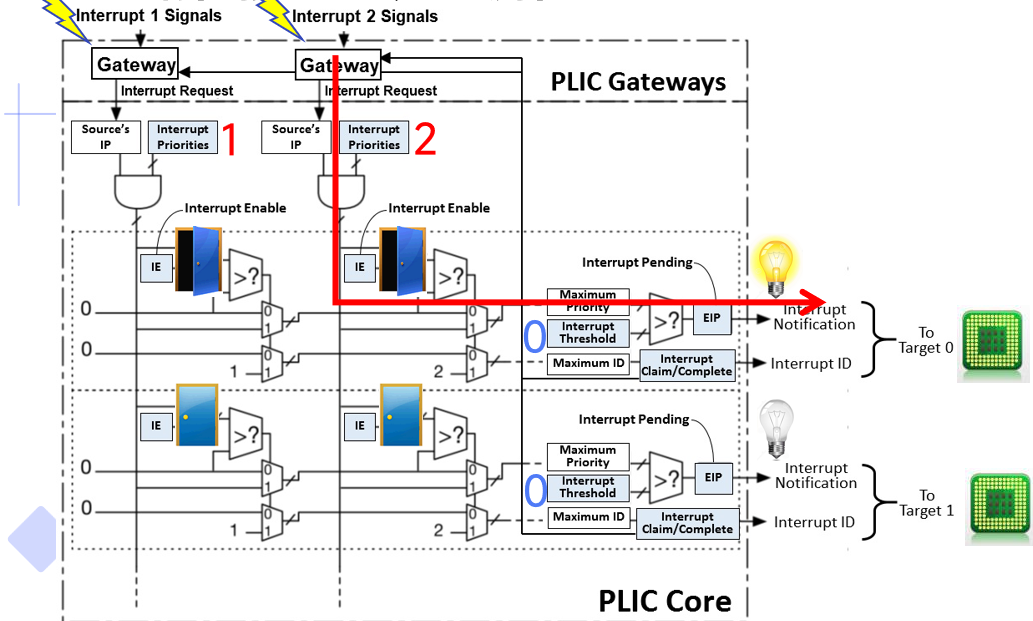


Figure 3. PLIC Operation Parameter Block Diagram

# PLIC 编程接口 - 处理流程

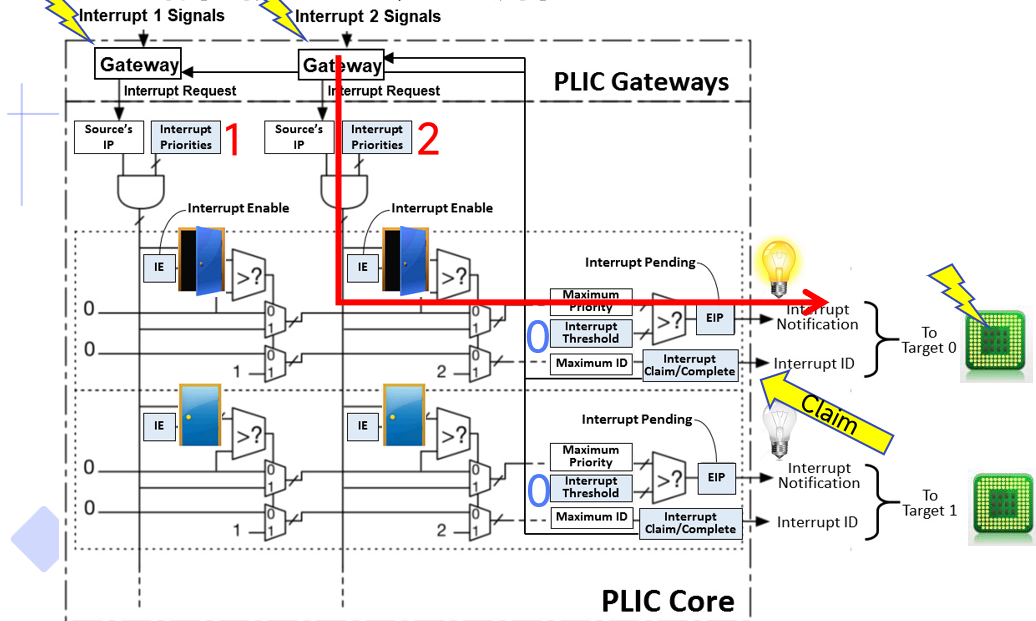


Figure 3. PLIC Operation Parameter Block Diagram

# PLIC 编程接口 - 处理流程

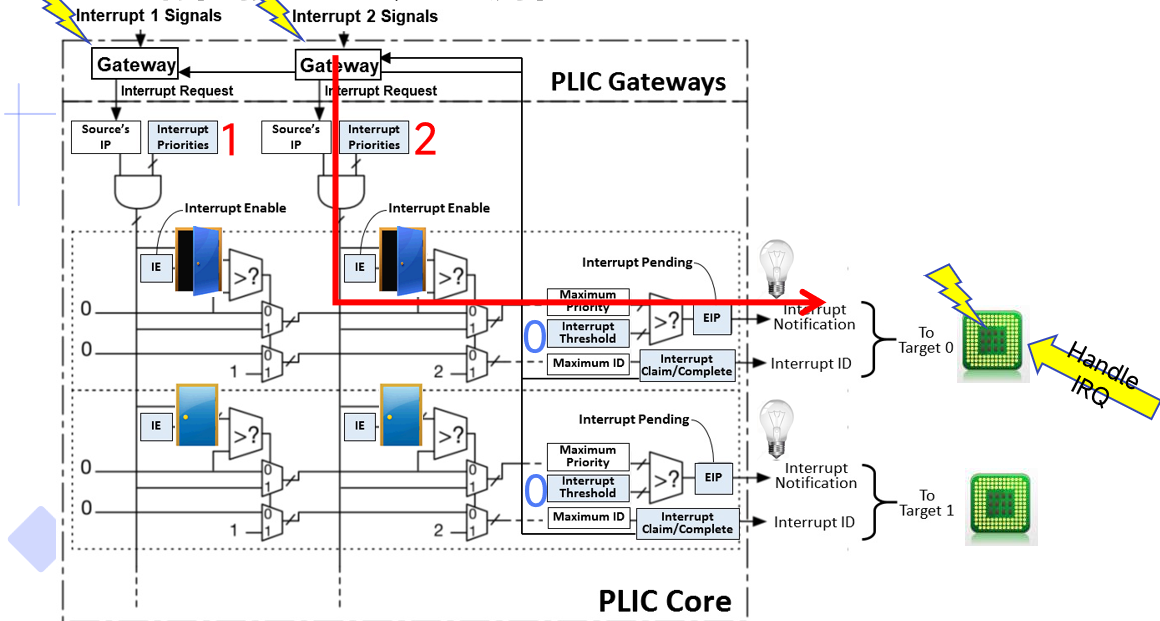


Figure 3. PLIC Operation Parameter Block Diagram

# PLIC 编程接口 - 处理流程

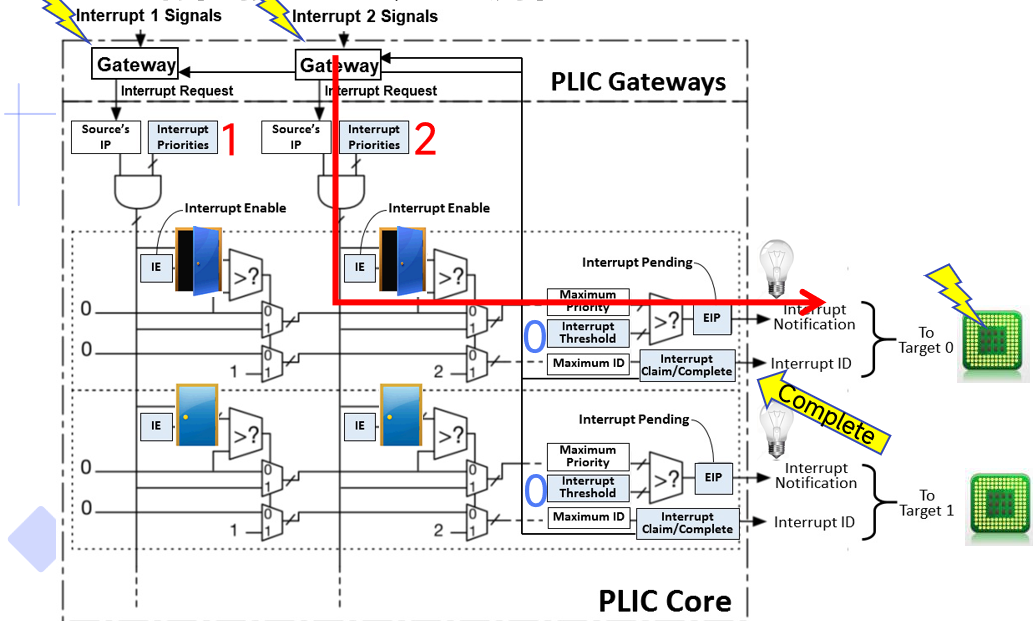


Figure 3. PLIC Operation Parameter Block Diagram



# PLIC 编程接口 - 处理流程

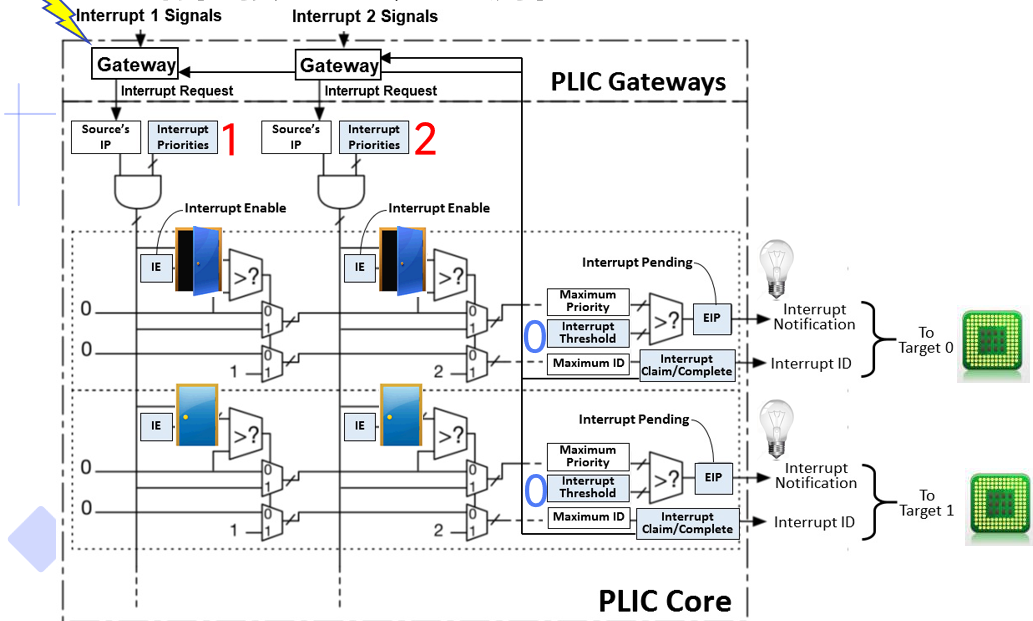


Figure 3. PLIC Operation Parameter Block Diagram

# PLIC 编程接口 - 处理流程

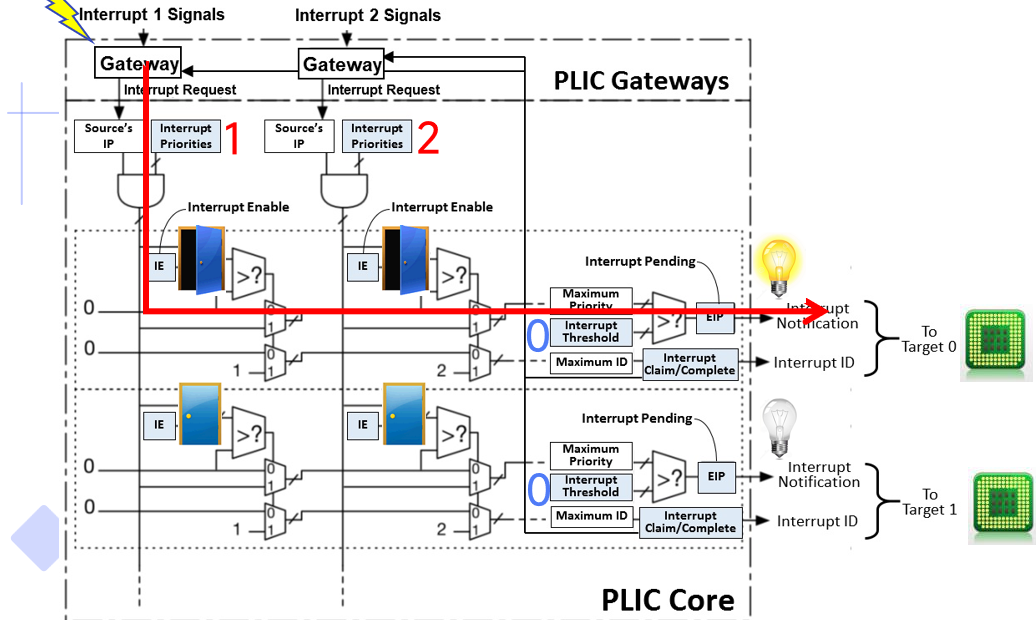


Figure 3. PLIC Operation Parameter Block Diagram



05

## 基于中断方式实现串口输入

# NS16550a 编程接口回顾

| A2 | A1 | A0 | READ MODE                   | WRITE MODE                          |
|----|----|----|-----------------------------|-------------------------------------|
| 0  | 0  | 0  | • Receive Holding Register  | • Transmit Holding Register         |
| 0  | 0  | 0  | N/A                         | • LSB of Divisor Latch when Enabled |
| 0  | 0  | 1  | N/A                         | • Interrupt Enable Register         |
| 0  | 0  | 1  | N/A                         | • MSB of Divisor Latch when Enabled |
| 0  | 1  | 0  | • Interrupt Status Register | • FIFO control Register             |
| 0  | 1  | 1  | N/A                         | • Line Control Register             |
| 1  | 0  | 0  | N/A                         | • Modem Control Register            |
| 1  | 0  | 1  | • Line Status Register      | N/A                                 |
| 1  | 1  | 0  | • Modem Status Register     | N/A                                 |
| 1  | 1  | 1  | • Scratchpad Register Read  | • Scratchpad Register Write         |

# NS16550a 编程接口回顾

| A2 | A1 | A0 | READ MODE                   | WRITE MODE                          |
|----|----|----|-----------------------------|-------------------------------------|
| 0  | 0  | 0  | • Receive Holding Register  | • Transmit Holding Register         |
| 0  | 0  | 0  | N/A                         | • LSB of Divisor Latch when Enabled |
| 0  | 0  | 1  | N/A                         | • Interrupt Enable Register         |
| 0  | 0  | 1  | N/A                         | • MSB of Divisor Latch when Enabled |
| 0  | 1  | 0  | • Interrupt Status Register | • FIFO control Register             |
| 0  | 1  | 1  | N/A                         | • Line Control Register             |
| 1  | 0  | 0  | N/A                         | • Modem Control Register            |
| 1  | 0  | 1  | • Line Status Register      | N/A                                 |
| 1  | 1  | 0  | • Modem Status Register     | N/A                                 |
| 1  | 1  | 1  | • Scratchpad Register Read  | • Scratchpad Register Write         |

# NS16550a 编程接口（中断接收）

| A2 | A1 | A0 | READ MODE                   | WRITE MODE                          |
|----|----|----|-----------------------------|-------------------------------------|
| 0  | 0  | 0  | • Receive Holding Register  | • Transmit Holding Register         |
| 0  | 0  | 0  | N/A                         | • LSB of Divisor Latch when Enabled |
| 0  | 0  | 1  | N/A                         | • Interrupt Enable Register         |
| 0  | 0  | 1  | N/A                         | • MSB of Divisor Latch when Enabled |
| 0  | 1  | 0  | • Interrupt Status Register | • FIFO control Register             |
| 0  | 1  | 1  | N/A                         | • Line Control Register             |
| 1  | 0  | 0  | N/A                         | • Modem Control Register            |
| 1  | 0  | 1  | • Line Status Register      | N/A                                 |
| 1  | 1  | 0  | • Modem Status Register     | N/A                                 |
| 1  | 1  | 1  | • Scratchpad Register Read  | • Scratchpad Register Write         |

# NS16550a 编程接口（中断接收）

| A2 | A1 | A0 | READ MODE                   | WRITE MODE                  |
|----|----|----|-----------------------------|-----------------------------|
| 0  | 0  | 0  | • Receive Holding Register  | • Transmit Holding Register |
| 0  | 0  | 0  | N/A                         |                             |
| 0  | 0  | 1  | N/A                         |                             |
| 0  | 0  | 1  | N/A                         |                             |
| 0  | 1  | 0  | • Interrupt Status Register |                             |
| 0  | 1  | 1  | N/A                         |                             |
| 1  | 0  | 0  | N/A                         | • Modem Control Register    |
| 1  | 0  | 1  | • Line Status Register      |                             |
| 1  | 1  | 0  | • Modem Status Register     |                             |
| 1  | 1  | 1  | • Scratchpad Register Read  | • Scratchpad Register Write |

```
// read and process incoming characters.
while(1){
 int c = uartgetc();
 if(c == -1)
 break;
 uartputc_sync(c);
 uartputc_sync('\n');
}
```

```
// read one input character from the UART.
// return -1 if none is waiting.
int uartgetc(void)
{
 if(ReadReg(LSR) & LSR_RX_READY){
 // input data is ready.
 return ReadReg(RHR);
 } else {
 return -1;
 }
}
```

# NS16550a 编程接口（中断接收）

| A2                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                 | A1 | A0 | READ MODE                   | WRITE MODE                          |                                            |    |    |    |                         |   |   |   |   |   |                                     |   |   |   |   |   |                             |   |   |   |   |   |                               |   |   |   |   |   |                                            |   |   |   |   |   |                             |                         |
|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----|----|-----------------------------|-------------------------------------|--------------------------------------------|----|----|----|-------------------------|---|---|---|---|---|-------------------------------------|---|---|---|---|---|-----------------------------|---|---|---|---|---|-------------------------------|---|---|---|---|---|--------------------------------------------|---|---|---|---|---|-----------------------------|-------------------------|
| 0                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                  | 0  | 0  | • Receive Holding Register  | • Transmit Holding Register         |                                            |    |    |    |                         |   |   |   |   |   |                                     |   |   |   |   |   |                             |   |   |   |   |   |                               |   |   |   |   |   |                                            |   |   |   |   |   |                             |                         |
| 0                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                  | 0  | 0  | N/A                         | • LSB of Divisor Latch when Enabled |                                            |    |    |    |                         |   |   |   |   |   |                                     |   |   |   |   |   |                             |   |   |   |   |   |                               |   |   |   |   |   |                                            |   |   |   |   |   |                             |                         |
| 0                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                  | 0  | 1  | N/A                         | • Interrupt Enable Register         |                                            |    |    |    |                         |   |   |   |   |   |                                     |   |   |   |   |   |                             |   |   |   |   |   |                               |   |   |   |   |   |                                            |   |   |   |   |   |                             |                         |
| 0                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                  | 0  | 1  | N/A                         | • MSB of Divisor Latch when Enabled |                                            |    |    |    |                         |   |   |   |   |   |                                     |   |   |   |   |   |                             |   |   |   |   |   |                               |   |   |   |   |   |                                            |   |   |   |   |   |                             |                         |
| 0                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                  | 1  | 0  | • Interrupt Status Register | • FIFO control Register             |                                            |    |    |    |                         |   |   |   |   |   |                                     |   |   |   |   |   |                             |   |   |   |   |   |                               |   |   |   |   |   |                                            |   |   |   |   |   |                             |                         |
| <table><tr><th>P</th><th>D3</th><th>D2</th><th>D1</th><th>D0</th><th>SOURCE OF THE INTERRUPT</th></tr><tr><td>1</td><td>0</td><td>1</td><td>1</td><td>0</td><td>LSR (Receiver Line Status Register)</td></tr><tr><td>2</td><td>0</td><td>1</td><td>0</td><td>0</td><td>RXRDY (Received Data Ready)</td></tr><tr><td>2</td><td>1</td><td>1</td><td>0</td><td>0</td><td>RXRDY (Received Data Timeout)</td></tr><tr><td>3</td><td>0</td><td>0</td><td>1</td><td>0</td><td>TXRDY (Transmitter Holding Register Empty)</td></tr><tr><td>4</td><td>0</td><td>0</td><td>0</td><td>0</td><td>MSR (Modem Status Register)</td></tr></table> |    |    |                             | P                                   | D3                                         | D2 | D1 | D0 | SOURCE OF THE INTERRUPT | 1 | 0 | 1 | 1 | 0 | LSR (Receiver Line Status Register) | 2 | 0 | 1 | 0 | 0 | RXRDY (Received Data Ready) | 2 | 1 | 1 | 0 | 0 | RXRDY (Received Data Timeout) | 3 | 0 | 0 | 1 | 0 | TXRDY (Transmitter Holding Register Empty) | 4 | 0 | 0 | 0 | 0 | MSR (Modem Status Register) | • Line Control Register |
| P                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                  | D3 | D2 | D1                          | D0                                  | SOURCE OF THE INTERRUPT                    |    |    |    |                         |   |   |   |   |   |                                     |   |   |   |   |   |                             |   |   |   |   |   |                               |   |   |   |   |   |                                            |   |   |   |   |   |                             |                         |
| 1                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                  | 0  | 1  | 1                           | 0                                   | LSR (Receiver Line Status Register)        |    |    |    |                         |   |   |   |   |   |                                     |   |   |   |   |   |                             |   |   |   |   |   |                               |   |   |   |   |   |                                            |   |   |   |   |   |                             |                         |
| 2                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                  | 0  | 1  | 0                           | 0                                   | RXRDY (Received Data Ready)                |    |    |    |                         |   |   |   |   |   |                                     |   |   |   |   |   |                             |   |   |   |   |   |                               |   |   |   |   |   |                                            |   |   |   |   |   |                             |                         |
| 2                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                  | 1  | 1  | 0                           | 0                                   | RXRDY (Received Data Timeout)              |    |    |    |                         |   |   |   |   |   |                                     |   |   |   |   |   |                             |   |   |   |   |   |                               |   |   |   |   |   |                                            |   |   |   |   |   |                             |                         |
| 3                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                  | 0  | 0  | 1                           | 0                                   | TXRDY (Transmitter Holding Register Empty) |    |    |    |                         |   |   |   |   |   |                                     |   |   |   |   |   |                             |   |   |   |   |   |                               |   |   |   |   |   |                                            |   |   |   |   |   |                             |                         |
| 4                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                  | 0  | 0  | 0                           | 0                                   | MSR (Modem Status Register)                |    |    |    |                         |   |   |   |   |   |                                     |   |   |   |   |   |                             |   |   |   |   |   |                               |   |   |   |   |   |                                            |   |   |   |   |   |                             |                         |
|                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                    |    |    |                             | • Modem Control Register            |                                            |    |    |    |                         |   |   |   |   |   |                                     |   |   |   |   |   |                             |   |   |   |   |   |                               |   |   |   |   |   |                                            |   |   |   |   |   |                             |                         |
|                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                    |    |    |                             | N/A                                 |                                            |    |    |    |                         |   |   |   |   |   |                                     |   |   |   |   |   |                             |   |   |   |   |   |                               |   |   |   |   |   |                                            |   |   |   |   |   |                             |                         |
|                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                    |    |    |                             | N/A                                 |                                            |    |    |    |                         |   |   |   |   |   |                                     |   |   |   |   |   |                             |   |   |   |   |   |                               |   |   |   |   |   |                                            |   |   |   |   |   |                             |                         |
| 1                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                  | 1  | 1  | • Scratchpad Register Read  | • Scratchpad Register Write         |                                            |    |    |    |                         |   |   |   |   |   |                                     |   |   |   |   |   |                             |   |   |   |   |   |                               |   |   |   |   |   |                                            |   |   |   |   |   |                             |                         |



# NS16550a 编程接口（中断接收）

| A2 | A1 | A0 | READ MODE                   | WRITE MODE |
|----|----|----|-----------------------------|------------|
| 0  | 0  | 0  | • Receive Holding Register  |            |
| 0  | 0  | 0  | N/A                         |            |
| 0  | 0  | 1  | N/A                         |            |
| 0  | 0  | 1  | N/A                         |            |
| 0  | 1  | 0  | • Interrupt Status Register |            |

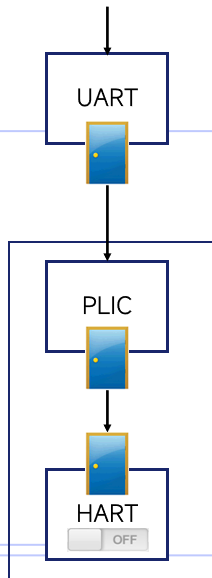
| P | D3 | D2 | D1 | D0 | SOURCE OF THE INTERRUPT                    |
|---|----|----|----|----|--------------------------------------------|
| 1 | 0  | 1  | 1  | 0  | LSR (Receiver Line Status Register)        |
| 2 | 0  | 1  | 0  | 0  | RXRDY (Received Data Ready)                |
| 2 | 1  | 1  | 0  | 0  | RXRDY (Received Data Timeout)              |
| 3 | 0  | 0  | 1  | 0  | TXRDY (Transmitter Holding Register Empty) |
| 4 | 0  | 0  | 0  | 0  | MSR (Modem Status Register)                |

1 1 1 • Scratchpad Register Read • Scratchpad

```
void uartintr(void)
{
 ReadReg(ISR); // acknowledge the interrupt

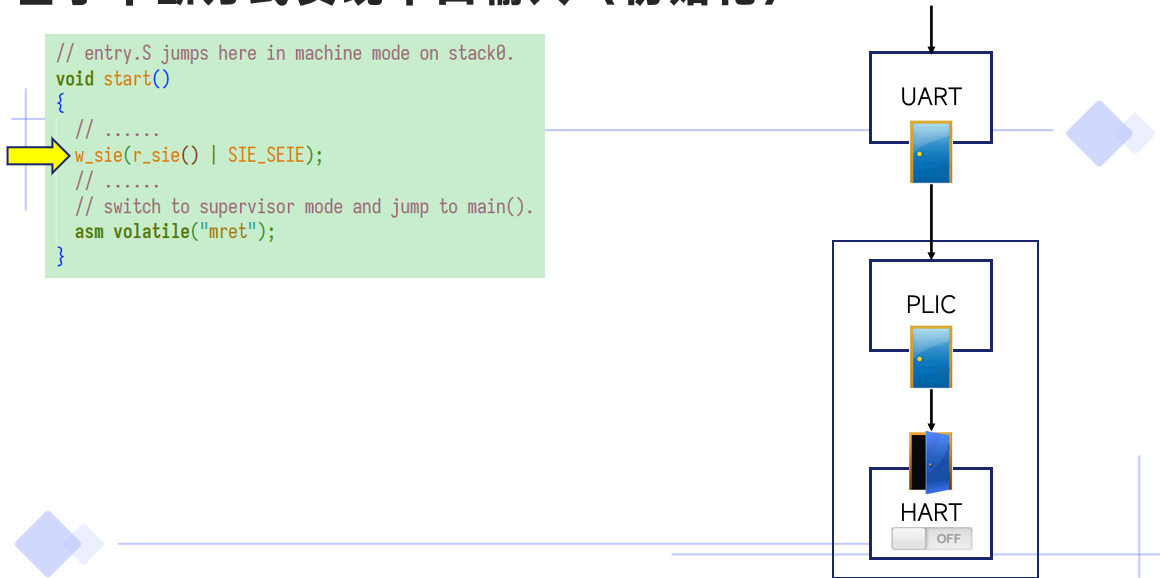
 // read and process incoming characters.
 while(1){
 int c = uartgetc();
 if(c == -1) break;
 // read one input character from the UART.
 // return -1 if none is waiting.
 int uartgetc(void)
 {
 if(ReadReg(LSR) & LSR_RX_READY){
 // input data is ready.
 return ReadReg(RHR);
 } else {
 return -1;
 }
 }
 }
}
```

# 基于中断方式实现串口输入（初始化）



# 基于中断方式实现串口输入（初始化）

```
// entry.S jumps here in machine mode on stack0.
void start()
{
 //
 w_sie(r_sie() | SIE_SEIE);
 //
 // switch to supervisor mode and jump to main().
 asm volatile("mret");
}
```

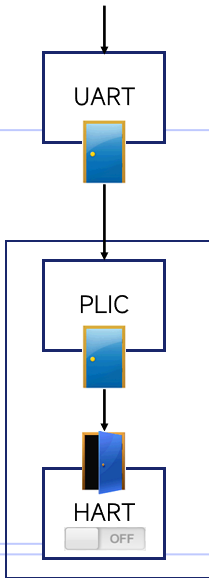


# 基于中断方式实现串口输入（初始化）

```
// entry.S jumps here in machine mode on stack0.
void start()
{
 //
 w_sie(r_sie() | SIE_SEIE);
 //
 // switch to supervisor mode and jump to main().
 asm volatile("mret");
}
```



```
void main()
{
 uartinit();
 //
 plicinit();
 plicinithart();
 //
 scheduler();
}
```



# 基于中断方式实现串口输入（初始化）

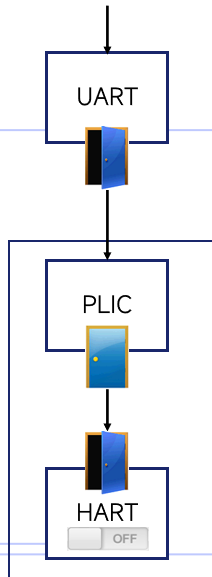
```
// entry.S jumps here in machine mode on stack0.
void start()
{
 //
 w_sie(r_sie() | SIE_SEIE);
 //
 // switch to supervisor mode and jump to main().
 asm volatile("mret");
}
```

```
void main()
{
```

```
 uartinit();
 //
 plicinit();
 plicinithart();
 //
 scheduler();
}
```

```
void uartinit(void)
```

```
{
 //
 // enable receive interrupts.
 WriteReg(IER, IER_RX_ENABLE);
}
```

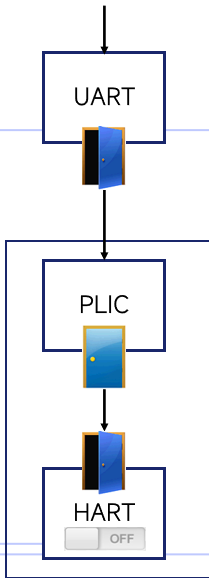


# 基于中断方式实现串口输入（初始化）

```
// entry.S jumps here in machine mode on stack0.
void start()
{
 //
 w_sie(r_sie() | SIE_SEIE);
 //
 // switch to supervisor mode and jump to main().
 asm volatile("mret");
}
```

```
void main()
{
 uartinit();
 //
 plicinit();
 plicinithart
 //
 scheduler();
}
```

```
void plicinit(void)
{
 // set desired IRQ priorities non-zero
 (uint32)(PLIC + UART0_IRQ*4) = 1;
}
```

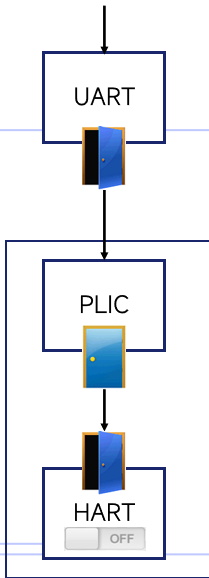


# 基于中断方式实现串口输入（初始化）

```
// entry.S jumps here in machine mode on stack0.
void start()
{
 //
 w_sie(r_sie() | SIE_SEIE);
 //
 // switch to supervisor mode and jump to main().
 asm volatile("mret");
}
```



```
void main()
{
 uartinit();
 //
 plicinit();
 plicinithart();
 //
 scheduler();
}
```

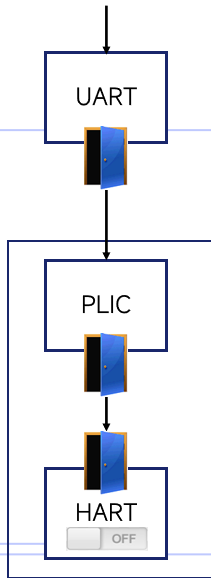


# 基于中断方式实现串口输入（初始化）

```
// entry.S jumps here in machine mode on stack0.
void start()
{
 //
 w_sie(r_sie() | SIE_SEIE);
 //
 // switch to supervisor mode and jump to main().
 asm volatile("mret");
}
```

```
void main()
{
 uartinit();
 //
 plicinit();
 plicinithart();
 //
 scheduler();
}
```

```
void plicinithart(void)
{
 int hart = cpuid();
 // set enable bits for this hart's S-mode
 // for the uart.
 (uint32)PLIC_SENABLE(hart) = (1 << UART0_IRQ);
 // set this hart's S-mode priority threshold to 0.
 (uint32)PLIC_SPRIORITY(hart) = 0;
}
```

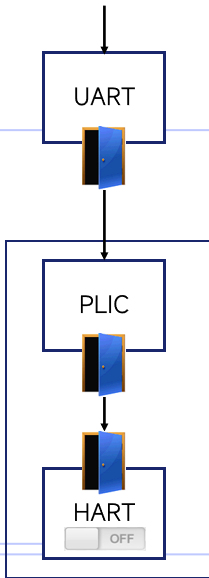




# 基于中断方式实现串口输入（初始化）

```
// entry.S jumps here in machine mode on stack0.
void start()
{
 //
 w_sie(r_sie() | SIE_SEIE);
 //
 // switch to supervisor mode and jump to main().
 asm volatile("mret");
}
```

```
void main()
{
 uartinit();
 //
 plicinit();
 plicinithart();
 //
 scheduler();
}
```



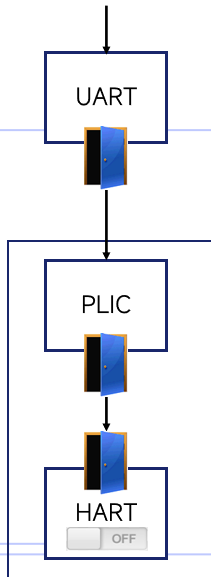
# 基于中断方式实现串口输入（初始化）

```
// entry.S jumps here in machine mode on stack0.
void start()
{
 //
 w_sie(r_sie() | SIE_SEIE);
 //
 // switch to supervisor mode and jump to main().
 asm volatile("mret");
}
```

```
void main()
{
 uartinit();
 //
 plicinit();
 plicinithart();
 //
 scheduler();
}
```

```
void scheduler(void)
{
 //
 // Enable interrupts on this processor.
 intr_on();
 //
 for(;;){
 //
 }
}
```

```
// enable device interrupts
static inline void
intr_on()
{
 w_sstatus(r_sstatus() | SSTATUS_SIE);
}
```



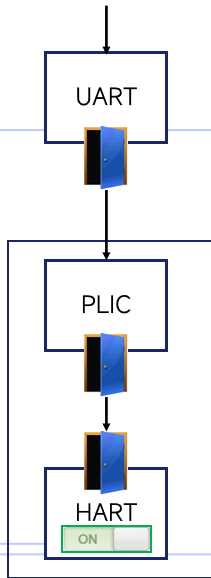
# 基于中断方式实现串口输入（初始化）

```
// entry.S jumps here in machine mode on stack0.
void start()
{
 //
 w_sie(r_sie() | SIE_SEIE);
 //
 // switch to supervisor mode and jump to main().
 asm volatile("mret");
}
```

```
void main()
{
 uartinit();
 //
 plicinit();
 plicinithart();
 //
 scheduler();
}
```

```
void scheduler(void)
{
 //
 // Enable interrupts on this processor.
 intr_on();
 //
 for(;;){
 //
 }
}
```

```
// enable device interrupts
static inline void
intr_on()
{
 w_sstatus(r_sstatus() | SSTATUS_SIE);
}
```



# 基于中断方式实现串口输入（中断处理）

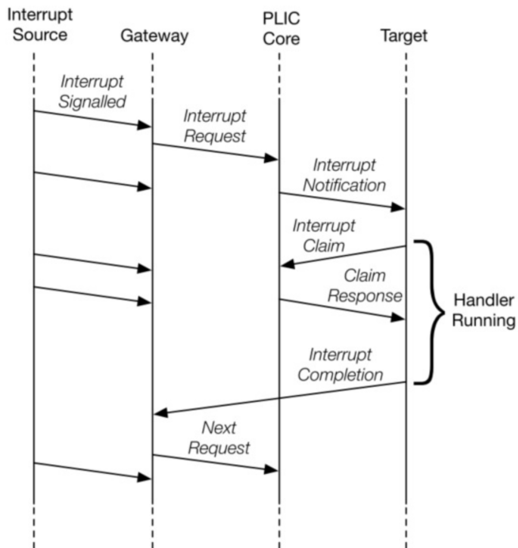
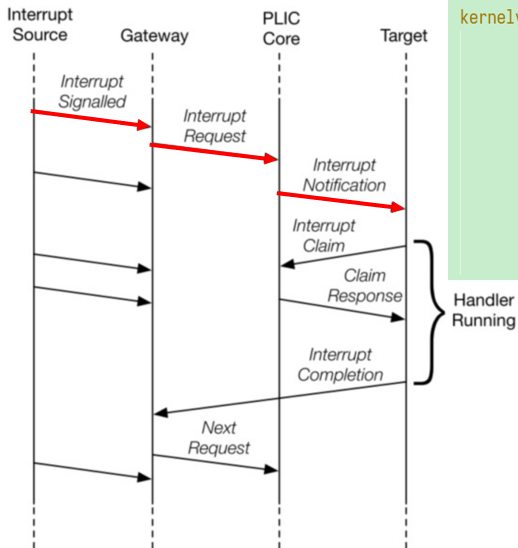


Figure 2. PLIC Interrupt Flow

# 基于中断方式实现串口输入（中断处理）

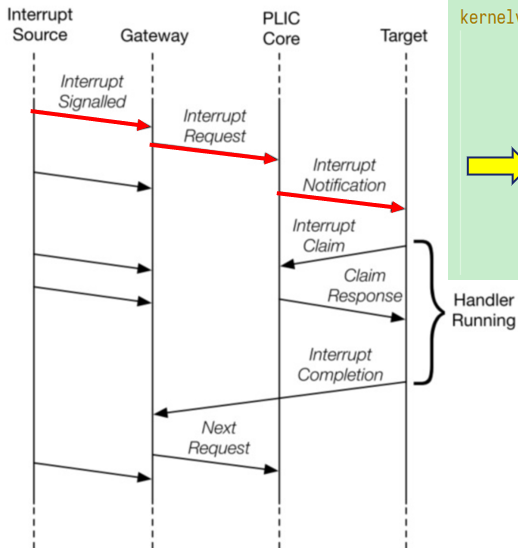


kernelvec:

```
make room to save registers.
addi sp, sp, -256
save caller-saved registers.
//
call the C trap handler in trap.c
call kerneltrap
restore registers.
//
return to whatever we were doing in the kernel.
sret
```

Figure 2. PLIC Interrupt Flow

# 基于中断方式实现串口输入（中断处理）



kernelvec:

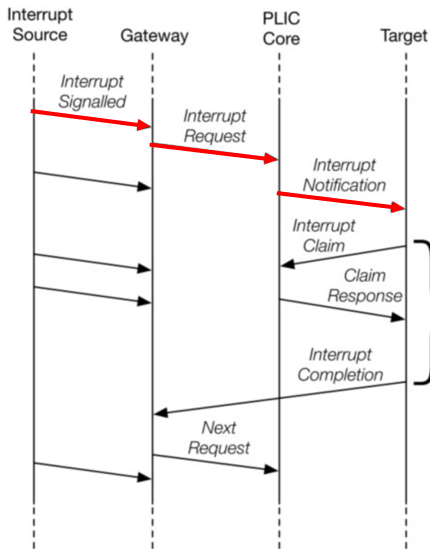
```
make room to save registers.
addi sp, sp, -256
save caller-saved registers.
//
call the C trap handler in trap.c
call kerneltrap
re
// .
re
sret
```

void kerneltrap()

```
{
 int which_dev = 0;
 //
 if((which_dev = devintr()) == 0){
 // interrupt or trap from an unknown source
 //
 panic("kerneltrap");
 }
}
```

Figure 2. PLIC Interrupt Flow

# 基于中断方式实现串口输入（中断处理）



kernelvec:

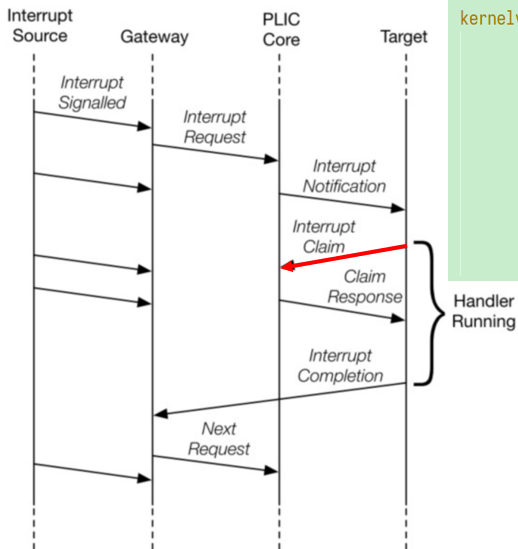
```
make room to save registers.
addi sp, sp, -256
save caller-saved registers.
//
call the C trap handler in trap.c
call kerneltrap
```

```
re
// .
re
sret

void kerneltrap()
{
 int which_dev = 0;
 //
 if((which_dev = devintr()) == 0){
 // interrupt or trap from an unknown source
 //
 panic("kerneltrap");
 }
}
```

Figure 2. PLIC Interrupt Flow

# 基于中断方式实现串口输入（中断处理）



kernelvec:

```
make room to save registers.
```

```
addi sp, sp, -256
```

```
save int devintr()
```

```
// {
```

```
call uint64 scause = r_scause();
```

```
call ke if(scause == 0x800000000000009L){
```

```
re // this is a supervisor external interrupt, via PLIC.
```

```
// . // irq indicates which device interrupted.
```

```
re int irq = plic_claim();
```

```
sret if(irq == UART0_IRQ){
```

```
uartintr();
```

```
} else if(irq){
```

```
printf("unexpected interrupt irq=%d\n", irq);
```

```
}
```

```
if(irq)
```

```
plic_complete(irq);
```

```
return 1;
```

```
} else {
```

```
return 0;
```

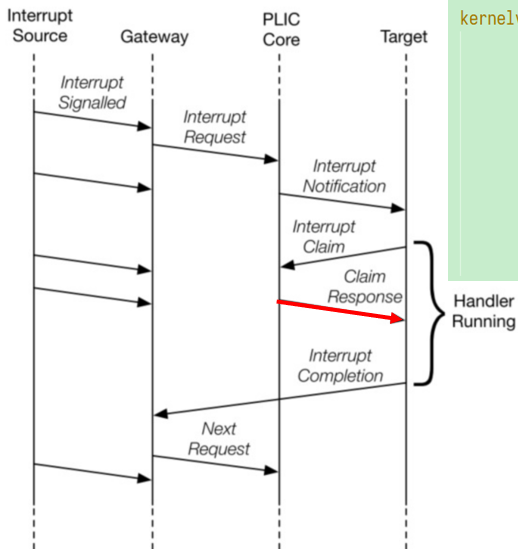
```
}
```

```
}
```

Figure 2. PLIC Interrupt Flow



# 基于中断方式实现串口输入（中断处理）



kernelvec:

```
make room to save registers.
```

```
addi sp, sp, -256
```

```
save registers in kernelvec
```

```
//
```

```
call kernelvec
```

```
call kernelvec
```

```
return to user mode
```

```
// .
```

```
return to user mode
```

```
sret
```

```
int devintr()
```

```
{
```

```
uint64 scause = r_scause();
```

```
if(scause == 0x8000000000000009L){
```

```
 // this is a supervisor external interrupt, via PLIC.
```

```
 // irq indicates which device interrupted.
```

```
 int irq = plic_claim();
```

```
 if(irq == UART0_IRQ){
```

```
 uartintr();
```

```
 } else if(irq){
```

```
 printf("unexpected interrupt irq=%d\n", irq);
```

```
 }
```

```
 if(irq)
```

```
 plic_complete(irq);
```

```
 return 1;
```

```
 } else {
```

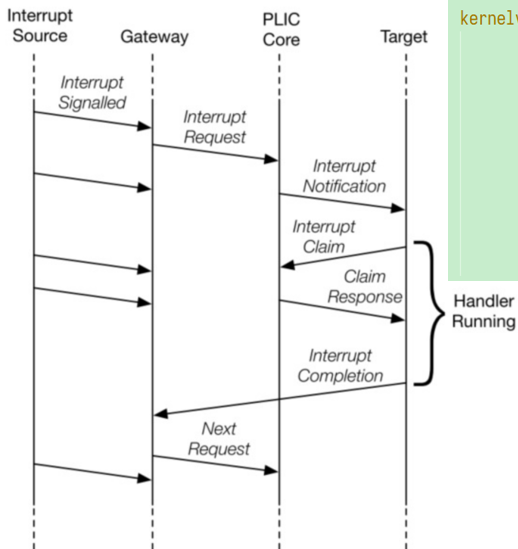
```
 return 0;
```

```
 }
```

```
}
```

Figure 2. PLIC Interrupt Flow

# 基于中断方式实现串口输入（中断处理）



kernelvec:

```
make room to save registers.
```

```
addi sp, sp, -256
```

```
save
```

```
//
```

```
call
```

```
call ke
```

```
re
```

```
// .
```

```
re
```

```
sret
```

```
int devintr()
```

```
{
```

```
uint64 scause = r_scause();
```

```
if(scause == 0x800000000000009L){
```

```
// this is a supervisor external interrupt, via PLIC.
```

```
// irq indicates which device interrupted.
```

```
int irq = plic_claim();
```

```
if(irq == UART0_IRQ){
```

```
uartintr();
```

```
} else void uartintr(void)
```

```
{
```

```
pri
```

```
}
```

```
ReadReg(ISR); // acknowledge the interrupt
```

```
// read and process incoming characters.
```

```
while(1){
```

```
int c = uartgetc();
```

```
} else
```

```
//
```

```
}
```

```
}
```

```
}
```

```
}
```

```
}
```

Figure 2. PLIC Interrupt Flow

# 基于中断方式实现串口输入（中断处理）

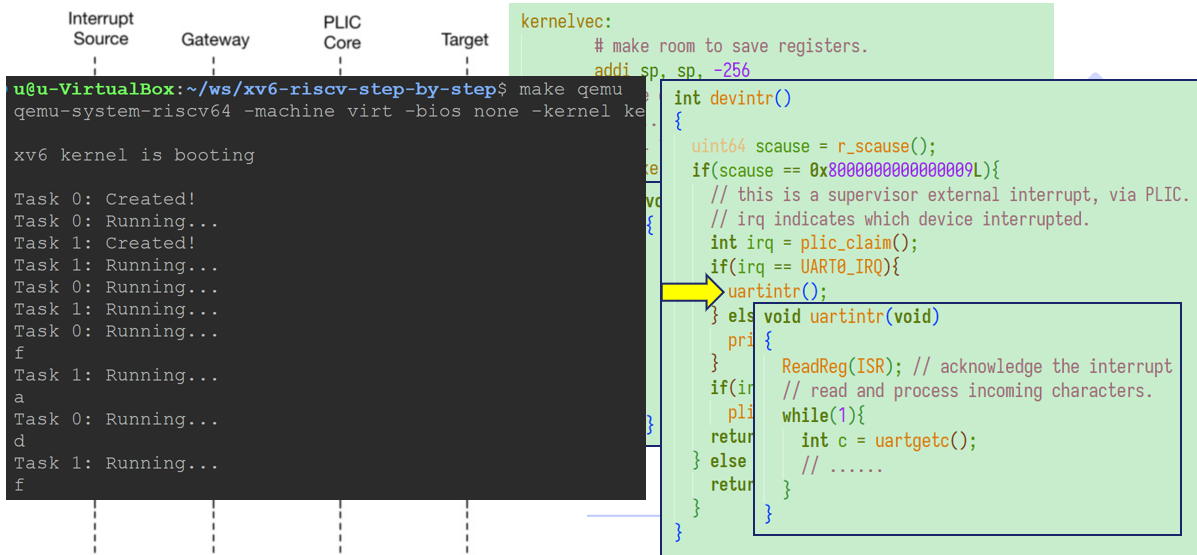
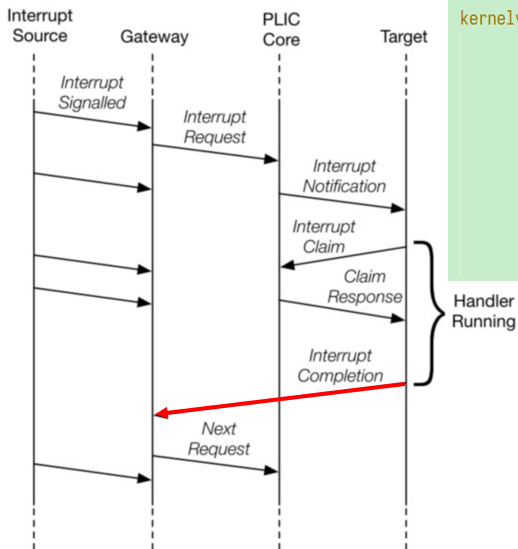


Figure 2. PLIC Interrupt Flow

# 基于中断方式实现串口输入（中断处理）



kernelvec:

```
make room to save registers.
```

```
addi sp, sp, -256
```

```
save registers
```

```
//
```

```
call kernelvec
```

```
call kernelvec
```

```
return to user mode
```

```
// .
```

```
return to user mode
```

```
sret
```

```
int devintr()
```

```
{
```

```
 uint64 scause = r_scause();
```

```
 if(scause == 0x8000000000000009L){
```

```
 // this is a supervisor external interrupt, via PLIC.
 // irq indicates which device interrupted.
```

```
 int irq = plic_claim();
```

```
 if(irq == UART0_IRQ){
```

```
 uartintr();
```

```
 } else if(irq){
```

```
 printf("unexpected interrupt irq=%d\n", irq);
```

```
 }
```

```
 if(irq)
```

```
 plic_complete(irq);
```

```
 return 1;
```

```
 } else {
```

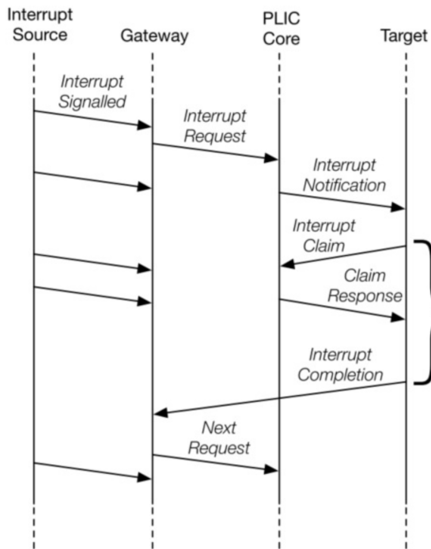
```
 return 0;
```

```
 }
```

```
}
```

Figure 2. PLIC Interrupt Flow

# 基于中断方式实现串口输入（中断处理）



kernelvec:

```
make room to save registers.
addi sp, sp, -256
save caller-saved registers.
//
call the C trap handler in trap.c
call kerneltrap
```

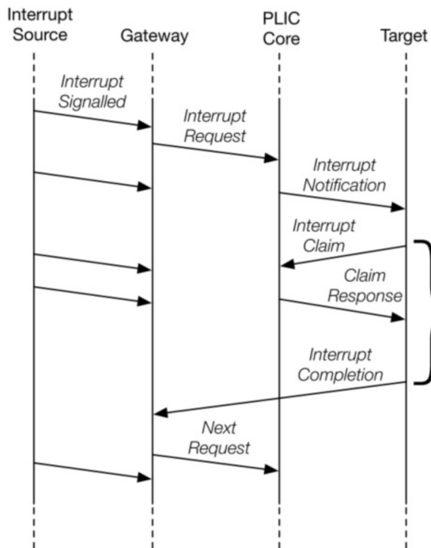
```
re
// .
re
sret
```

```
void kerneltrap()
```

```
{
 int which_dev = 0;
 //
 if((which_dev = devintr()) == 0){
 // interrupt or trap from an unknown source
 //
 panic("kerneltrap");
 }
}
```

Figure 2. PLIC Interrupt Flow

# 基于中断方式实现串口输入（中断处理）

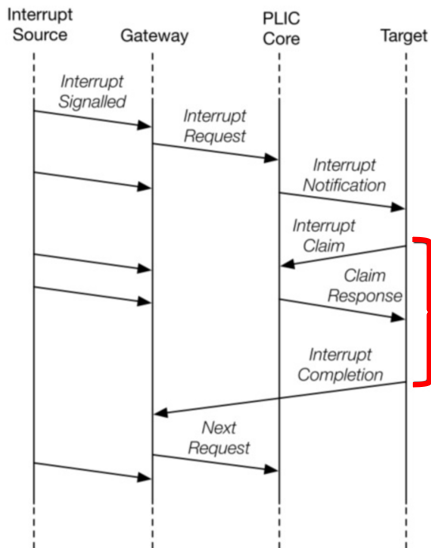


kernelvec:

```
make room to save registers.
addi sp, sp, -256
save caller-saved registers.
//
call the C trap handler in trap.c
call kerneltrap
restore registers.
//
return to whatever we were doing in the kernel.
sret
```

Figure 2. PLIC Interrupt Flow

# 基于中断方式实现串口输入（中断处理）



kernelvec:

```
make room to save registers.
addi sp, sp, -256
save caller-saved registers.
//
call the C trap handler in trap.c
call kerneltrap
restore registers.
//
return to whatever we were doing in the kernel.
sret
```

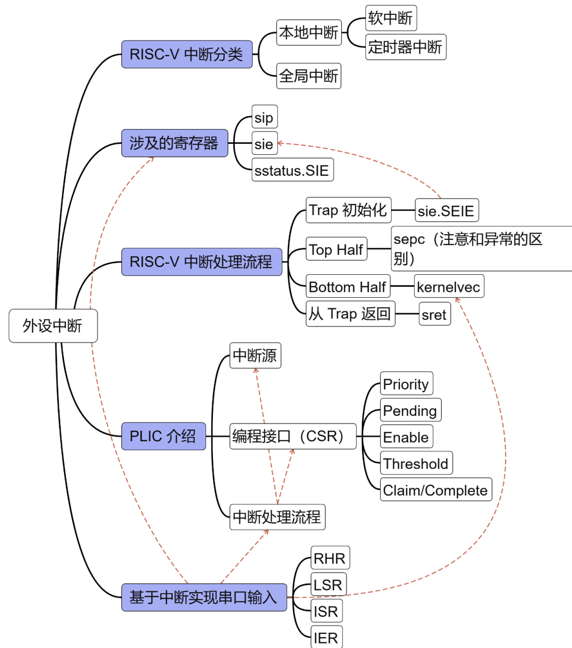
Figure 2. PLIC Interrupt Flow



## 本章总结

---





# 谢谢

